

C语言与程序设计

The C Programming Language

第7章 数组

王多强

1097412466

dqwang@hust.edu.cn

华中科技大学计算机学院

- 前面给出了变量的基本定义语句，如：

```
int x, y;
```

```
char c;
```

这样的变量是**简单变量**：每个变量只能表示单个的基本数据。

- 如果是一组数怎么办？

如一个**数列**包含了10个整数，定义怎样的变量表示？

- ◆ 独立变量： `int x1, x2, x3, x4, x5, x6, x7, x8, x9, x10;` **X**
- ◆ 定义数组： `int x[10];` **✓**

集合数据

本章介绍:

- 一维数组的声明、初始化、使用、运算、数组作为函数参数等;
- 字符数组、字符串, 以及字符串处理函数;
- 多维数组的声明、使用、存储结构、初始化;
- 介绍一些基本算法: 冒泡排序、矩阵加运算、矩阵乘运算、二分查找。

7.1 数组概述

例：一个班有30名学生，输入全班同学高数课程的成绩并计算平均成绩，如何处理？

方式一：定义30个整型变量，分别保存30名同学的高数成绩

```
int i1,i2,i3,i4,i5,i6,i7,i8,i9,i10, i11,i12,i13,i14,i15,  
    i16,i17,i18,i19,i20, i21,i22,i23,i24,i25, i26,i27,  
    i28,i29,i30;
```

试想一下，如何求和并计算平均成绩呢？

◆ 使用麻烦

◆ 扩展性差（如果有50、100个同学该怎么办？）！

方式二：定义一种特殊的数据结构——**数组**，来表示30个数据的集合，这样可以方便地保存30名同学的成绩并可利用**循环的方式**简洁地进行相关处理：

```
int a[30];
```

```
for(i=0;i<30;i++)
```

```
    scanf("%d", &a[i]);
```

```
sum = 0;
```

```
for(i=0;i<30;i++)
```

```
    sum = sum + a[i];
```

```
avg = sum/30;
```

```
/*定义含有30个元素的整型数组a*/
```

```
/*利用循环对数组进行运算*/
```

```
/*利用数组名[下标]的方式访问数组中的元素*/
```

```
/*求和*/
```

```
/*求平均分*/
```

7.2 一维数组的声明和使用

7.2.1 一维数组的声明

一维数组：只有**一个下标**的数组称为**一维数组**。

一维数组声明的基本形式：

类型说明符 数组名[常量表达式];

如： **int array[5];**

其中，

- ◆ **int**：数组的类型，**指数组中数据元素的数据类型**；
- ◆ **array**：数组的名子；
- ◆ **[5]**：说明数组的大小，即array数组中有5个元素。

一维数组声明的一般形式：

[存储类型说明符] [类型修饰符] 类型说明符 数组名[常量表达式]={初值表};

其中，

◆ 存储类型说明符（可选）：

- **extern**：用于声明外部数组（类似外部变量的声明）
- **static**：用于声明静态数组（类似静态变量的声明）

■ 类型修饰符（可选）：

- **const**：用于声明常量型数组
- **volatile**：用于声明易失性数组

[存储类型说明符] [类型修饰符] **类型说明符 数组名** [常量表达式] = {初值表};

◆ **类型说明符** (必需) :

- 用于说明数组中**数据元素的数据类型**，简称“**数组的类型**”；
- 类型说明符可以是int、char等简单数据类型，

如: **int** x[10];

char s[10];

分别称为**整型数组**，**字符型数组**

- 也可以是**结构**、**指针**等复杂数据类型，分别称为**结构数组**、**指针数组**等(第8、9章讲述)。
- 注: **一个数组中的所有元素具有相同的数据类型。**
即: **数组是同类型元素的有限序列。**

[存储类型说明符] [类型修饰符] **类型说明符 数组名[常量表达式]**={初值表};

◆ **数组名** (必需) :

- **数组的名称**, 数组名必须是一个**合法的标识符**。

◆ **[]** (必需) :

- 这里的**[]**称为**下标操作符**, 跟在数组名后, 定义数组时用于将数组名说明为数组;
- 一维数组只有一个下标。

数组的维数: 标识数组中的一个基本元素所需要使用的**下标的个数**称为**数组的维数**。

- ◆ **一维数组**: 只用一个下标的数组;
- ◆ **二维数组**: 要用二个下标的数组, 三维、四维数组依次类推;
- ◆ **多维数组**: 一般称二维以上的数组为多维数组。

[存储类型说明符] [类型修饰符] 类型说明符 数组名 **[常量表达式] = {初值表};**

◆ **常量表达式**（可选）：

- 定义数组时指定数组的大小，即数组中**最多**可以存放数据元素的个数。
- 必须为常量表达式，这样编译时就可计算值的大小，从而以明确数组的大小。

◆ **= {初值表}**（可选）：

- 用于创建数组时给数组中的元素赋初始值。
- **{ }**：初值表的界定符，里面列出初始值。
在有多多个初值时，初值之间用逗号隔开。

◆ **最后的分号 (;) 不可少。**

例7.1 一维数组的声明:

```
int score[30];
```

```
char code[10];
```

```
float length[100];
```

或先定义**整型符号常量**，然后用之声明数组：

```
#define SIZE 30
```

```
int score[SIZE];
```

```
char name[SIZE*5];    /*SIZE*5 是常量表达式*/
```



都是常量参与运算的表达式称为**常量表达式**。
常量表达式在编译阶段就可确定其值。

◆ **数组的大小**：指数组中可存放的数据元素的**最大个数**；

如，`int x[10]`；表示整型数组x里面**最多**可以存放10个元素。

- C语言要求：数组的大小在定义时指定，并且一经指定就不能改变；
- 标准C要求：用**常量表达式**指定数组的大小，以便在编译阶段就可以确定数组的大小。

如：以下数组声明在C89中是错误的：

```
unsigned int size;  
scanf("%d", &size);  
char str[2*size];
```



注：**数组的大小**是指数组中可存放数据元素的最大个数，数组里的实际元素数不能超过这个值，否则产生“溢出”错误。

■ 可变数组:

在C99及C11标准中，允许用**有具体值的变量**作为长度说明定义数组，这种数组叫**可变数组**。如：

```
unsigned int n=10;
```

```
buffer[n];
```

或

```
unsigned int size;
```

```
scanf("%d", &size);
```

```
char str[2*size];
```

/* 注：在C99及C11标准中合法，但在之前的版本中不合法*/

关于可变数组更多的说明请参见教材P174

声明静态数组、外部数组

1) **静态数组**：用存储类型修饰符**static**声明的数组

如：**static int y[10];**

◆ **静态数组中的每一个元素都是静态数据。**

2) **外部数组**：即**全局数组**，存储类型是**extern**

但：和前面讲过的简单外部变量一样，定义外部数组时不要用extern，只有做外部声明时才用extern。

如：**extern double s[2];**

7.2.2 一维数组元素的引用

数组是**同类型**元素的有限序列：

- 1) 数组中的每个元素可视为一个独立变量，它们只是“被编排”在一个数组里；
- 2) 数组中的**每个元素都可以单独访问**，可以独立参与该类型数据允许参与的所有运算；
- 3) 数组中的元素在数组中是**按位置顺序有序排列**的，用**下标**标注其位置，并如同数学里“**数列元素**”的用法：**数列名**_{下标}

如 $a_1, a_2, \dots, a_n$

C语言中对数组元素的引用也用“**数组名+下标**”的方式，但其形式是：**数组名[下标表达式]**

数组元素引用的一般形式：

数组名[下标表达式]

其中，

- ◆ **数组名**：是事先已定义好的数组的数组名；
- ◆ **[]**：数组**下标界定符**，优先级为1级的括号运算符；
- ◆ **下标表达式**：整型表达式，值为整型，但可以包含变量。

如，设有数组定义：**int a[5] , j=2;**

则数组a的5个元素可表示为：

a[0], a[1], a[2], a[3], a[4]

或：**a[j+2] 、 a[++j] 、 a[j--]、 a[5*j-7]**

关于数组的下标:

- ◆ C语言规定, 数组的下标从0开始;
- ◆ 大小为N的一维数组, 下标的合法取值范围是0 ~ N-1。

合法的下标值必须在 **[0 ~ N-1]** 范围内, 超出这个范围称为 **下标越界**。下标越界对程序是危险的, 可能会造成系统崩溃。

```
int a[5] , j=2;
```

- **a[0]、a[1]、a[2]、a[3]、a[4]、a[j+2]、a[++j]、a[j--]、a[5*j-7]** 合法。✓
- 而 **a[-1]、a[5]、a[j-3]、a[2*j+1]** 错误。✗

特别提醒：

C语言不主动检查下标越界的情况，也就是对越界的下标，C语言也不报错，直到出现异常终止程序的执行。这对使用数组留下了安全隐患。

越界访问数组会产生不可预计的后果，甚至造成系统崩溃，所以要特别注意。

如果下标越界了，如何理解对**越界下标**所对应的“**元素**”的引用呢？如 **a[-1]**是什么意思？

例7.3 分析下面程序段的问题并改正

```
int i, b[8];  
for(i=0; i<=8; i++)  
    printf("%d", b[i]);
```

分析：对于本例，合法的下标取值范围是**0~7**，而不是0~8。

更正：

或

```
int i, b[8];  
for(i=0; i<8; i++)  
    printf("%d", b[i]);
```

```
int i, b[8];  
for(i=0; i<=7; i++)  
    printf("%d", b[i]);
```

7.2.3 一维数组的初始化

在定义数组的时候，用**初值表**对数组中的元素赋初值叫**数组的初始化**。

初值表：由一对**花括号**界定，表中列出元素初值，当有多个初值的时候用逗号隔开。

如：`int x[5]={0,1,2,3,4};`

◆ 初始赋值的方式：

编译器按照初值表中初值的顺序，依次赋给数组的第一个元素（下标为0的元素）、第二个元素（下标为1的元素）、...、直到最后一个元素（下标为N-1的元素）。如上例，初始化后，x的5个元素的值分别为：

`x[0]=0, x[1]=1, x[2]=2, x[3]=3, x[4]=4`

初始化数组的注意事项:

(1) 初始化时可以指定数组的长度: 如: `int x [5]={0,1,2,3,4};`

也可以不指定数组的长度: 如: `int y[ ]={1,2,3,4,5,6,7,8};`

◆ 此时编译器根据初值个数确定数组的大小。

y被初始化成含有8个元素的整型数组。

(2) 如果指定数组的长度, 则初值表中初值的个数必须小于等于所说明的数组的长度。

如: `int x[5]={0,1,2,3,4,5};` ✗ /*错误, 越界*/

当初值的个数小于数组的长度时, 只能缺省后面连续元素的

初值。如: `int z[10]={0,1,2,3,4};` ✓ /*只对数组的前5个元素赋初值*/

而: `int u[9]={ , 1, , , 2};` ✗ /*错误*/

(3) 当初值表中初值的个数少于数组长度时，数组中前面的元素用初值表中的数据初始化，而**后面没有指定初值的元素被自动初始化为0**。

如： `int z[10]={0,1,2,3,4};`

- ◆ 数组的前5个元素`z[0]~z[4]`依次赋初值0~4，而后面的5个元素`z[5]~z[9]`被自动置为0。

应用：定义一个数组，将其中的元素全部清零

`int y[10]={0};`

- C99增加了一个新特性：**指定初始化器**。利用该特性可以初始化指定的数组元素。

(1) `int a[6]={[5]=2};`

- ◆ 将a[5]初始化为2，前几个元素没有初始化，由编译器会自动把它们设置为0。

(2) `int a[8]={[5]=2,3,4};`

- ◆ 这个时候a[5]初始化为2，后面将a[6]就初始化3，a[7]就初始化4，其余元素被编译器置为0。

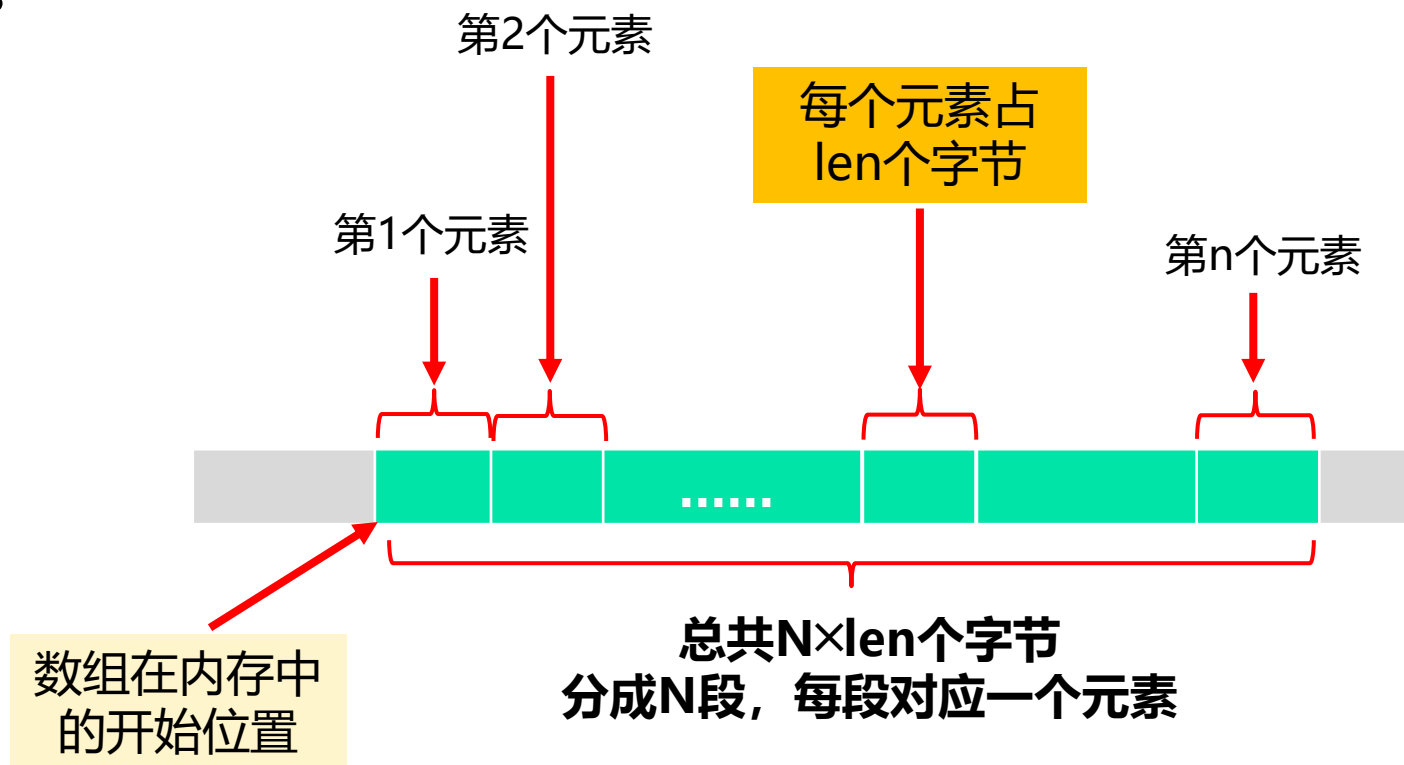
(3) `int a[ ]={1,[6]=2,3}`

- ◆ 这个时候a[0]初始化为1，后面将a[6]就初始化2，接着将a[7]就初始化3，其余元素被编译器置为0。最后数组长度设置为8。

(4) 如果再次初始化之前指定的元素，那么最后的初始化将会取代之前的初始化。（参见教材P180）

7.2.4 一维数组的存储

编译器为数组在内存中分配**连续的存储单元**存储数组中的元素：如果数组的大小是 **N** ，类型长度是 **len** ，则总共分配 **$N * len$** 个字节。这些字节**空间位置连续**，且**被分成 N 段**，每段存储一个数组元素。



数组的地址和数组元素的地址

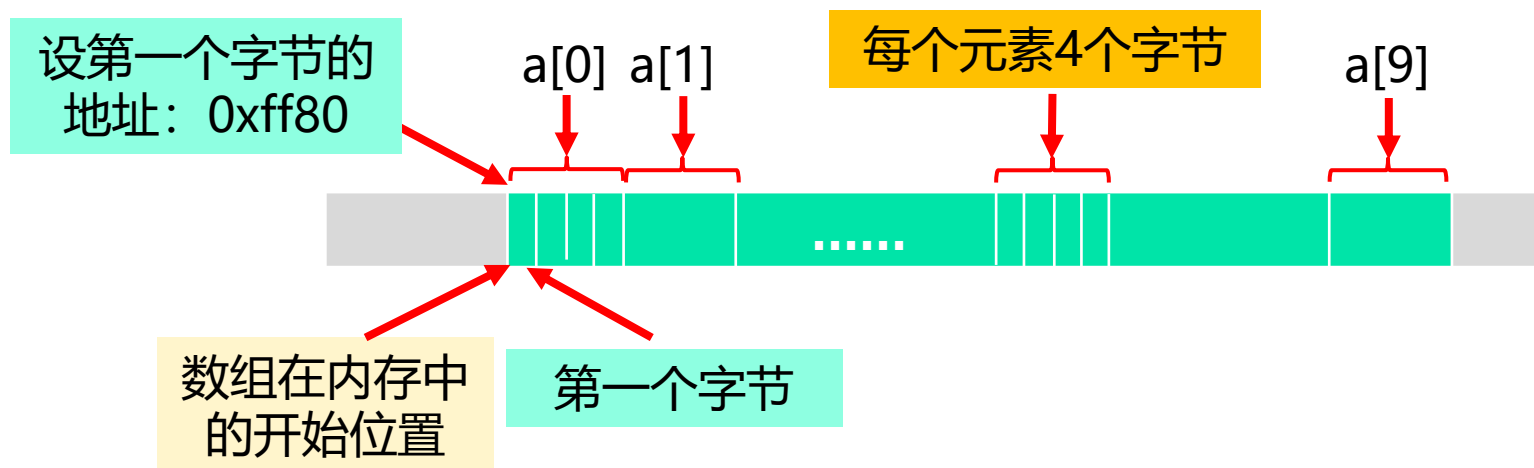
内存中每个字节都有地址，存在其中的每个变量也有地址。

(变量所占空间第一个字节的地址称为**变量的地址**)

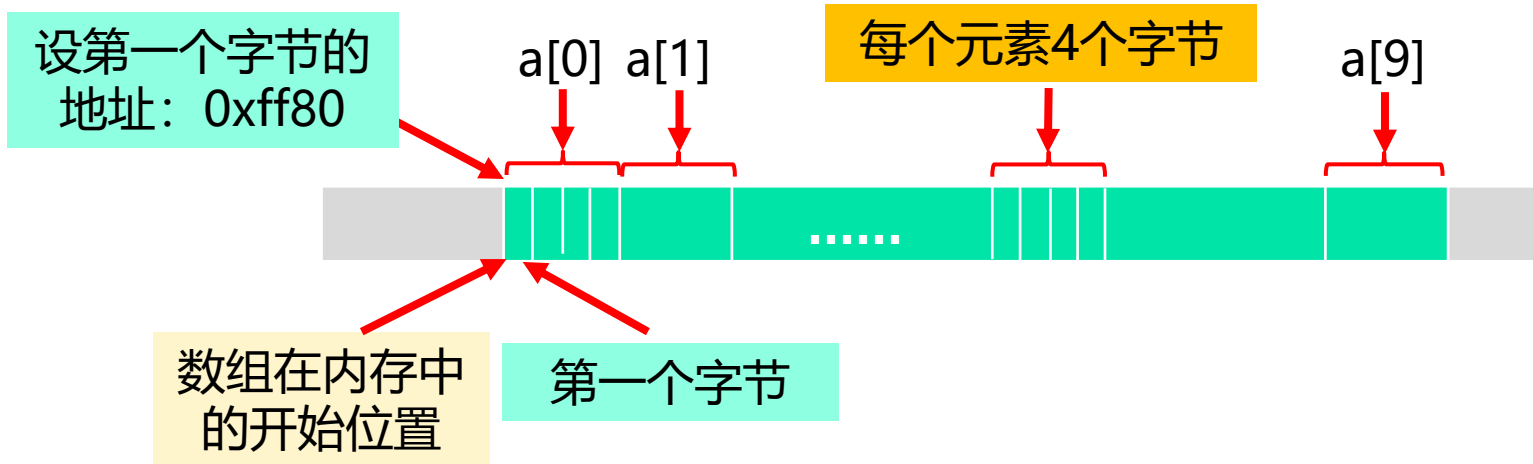
(1) 数组的地址

数组在内存中占有连续的字节空间，其中**第一个字节的地址**代表**数组的起始地址**，简称为**数组的地址**。

例：设有整型数组**int a[10];** (每个元素占4个字节，总共分配40个字节)



int a[10];



C语言约定: **数组名的值就是数组的地址值。**

如本例, a的值就是**0xff80**。

若 **printf("%x", a);** 则输出: **0xff80**

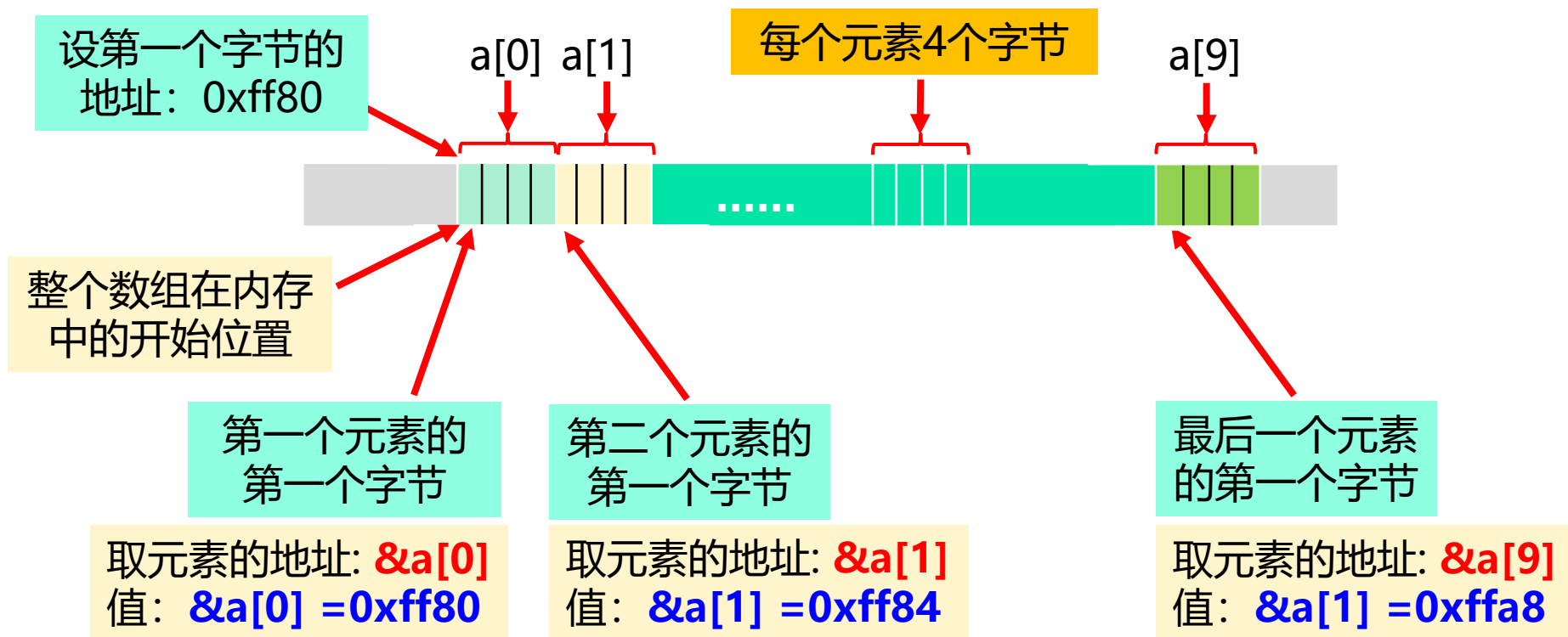
且, **数组名被视为地址常量**, 不能被修改。

如: **name=0xffa0;** **×**

(2) 数组元素的地址

数组中的每个元素也有实际的存储空间，**每个元素所占字节空间的第一个字节的地址是元素的起始地址**，简称为**数组元素的地址**。可以用“&”运算取元素的地址。

例：设有整型数组 `int a[10];`（每个元素占4个字节，总共分配40个字节）



◆ 关于数组及其元素的地址

(1) **只能对数组元素取地址，不能用&取数组名的地址。**

◆ 数组名本身的值就是数组的地址，如：**&a** ❌

(2) 整个数组的第一个字节也是第一个元素的第一个字节，所以**数组名的地址值和数组第一个元素的地址值相等。**

◆ 对一维数组，不仅数组名代表的地址值和数组第一个元素的地址值相等，而且它们的“**地址类型**”也相同；

◆ 但对多维数组而言，二者仅值相等，类型却不同。

所以**不能简单地认为二者等同（见后）。**

例7.5 以下程序输出一个数组的数组名的值、数组中各元素的值和地址，观察它们之间的关系

```
#include "stdio.h"
```

```
#define SIZE 3
```

```
int main(void)
```

```
{
```

```
int a[SIZE]={1, 3, 5}, k;
```

```
printf("the value of a is 0x%x\n",a);
```

```
for(k=0;k<SIZE;k++)
```

```
printf("k=%d\ta[%d]=%d\t addr=0x%x\n", k, k, a[k], &a[k]);
```

```
}
```

地址	数组元素
0x22ff30	1
0x22ff34	3
0x22ff38	5

一维数组a的物理存储示意图

设32位机，1个int变量占4个字节

运行结果：

the value of a is 0x22ff30

相等

k=0 x[0]=1

addr=0x22ff30

k=1 x[1]=3

addr=0x22ff34

k=2 x[2]=5

addr=0x22ff38

要注意数组元素的值和地址的不同：

- ◆ **元素的值**：指数组元素存储单元中的内容，用**数组名[下标]**的方式引用，如 `a[0]`、`a[i]`。
- ◆ **元素的地址**：指数组元素在内存中的存储单元的起始地址，要用**&**运算符取元素的地址，如 `&a[0]`、`&a[i]`

7.2.5 一维数组的运算

数组运算的基本要点：

- 1) 数组中的每个元素都可视为一个独立变量，可以参与其类型允许的所有运算。
- 2) 但C语言**不支持数组的整体运算**，**数组运算最后都要归结到对数组元素个体的操作上。**

如，设有三个数组定义：`int x[3]={1,2,3}, y[3]={4,5,6};`
`int z[3];`

正确的运算方式：

`z[0]=x[0]+y[0];`

`z[1]=x[1]+y[1];`

`z[2]=x[2]+y[2];`

若：`z=x+y;` ❌

非法操作，编译时给出错误提示：

“cannot add two pointers”

利用循环处理数组很方便：

```
int x[3]={1, 2, 3}, y[3]={4, 5, 6}, z[3];  
  
int i;  
  
for(i=0; i<3; i++) {  
    z[i]=x[i]+y[i];  
    printf("%d", z[i]);  
}
```

- ◆ **利用循环处理数组是数组最典型的用法。** 如果不使用循环，就失去了定义数组的意义了。

7.2.6 一维数组作为函数参数

(1) 函数的参数可以是**数组类型**的。

如： `void bubble(int a[ ], int n)`

这里，

- ◆ 数组形参简称“**形式数组**”；
- ◆ 形式数组的“**[]**”必不可少，其作用是**将一个参数名说明为数组类型**，而不是简单变量。
- ◆ **[]**中不用写数组的大小（写了也不起作用）。
 - 那怎么知道数组中有几个元素呢？
用另外一个参数（如这里的 `int n`）**显式地说明形式数组中元素的个数**。

(2) 调用：在主调函数中用**实参数组的数组名**作为实参。

如： `void bubble(int a[ ], int n)`

设， `int x[3]={1,2,3};`

调用： `bubble(x, 3);`

形参数组

指示形参数组的大小

实参数组名

给出实参数组的大小

C语言的参数传递规则是“**值传递**”，那么实参数组和形参数组结合时，传递的是什么呢？是不是将实参数组中的元素都复制给了形参数组呢？否！

传递的是实参数组的地址，将该地址复制给了形参数组名

(3) 实参数组至形参数组的参数传递

形参数组

如: `void bubble(int a[ ], int n)`

设, `int x[3]={1,2,3};`

调用: `bubble(x, 3);`

实参数组名

x数组的起始地址
设为0xff80



① 调用时, 为形式数组名开辟存储单元:
存放地址数据, 一种
无符号整数, 4个字节

形式数组名a的存储空间, 4个字节

② 把x的值 (x数组的起始地址) 复值给形参数据名

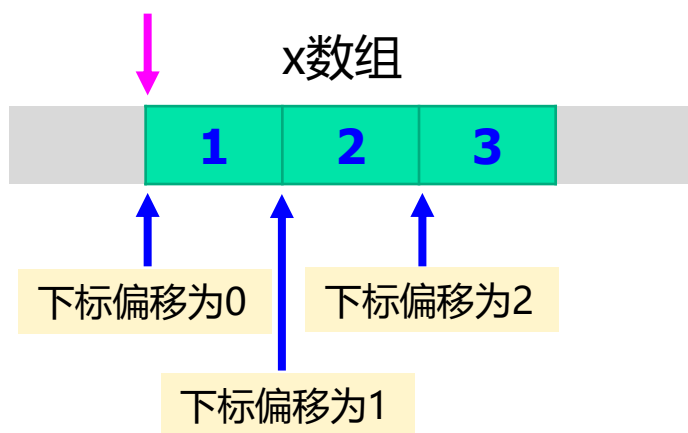
形参数组的参数传递 “址传递”

- ◆ 对数组元素引用形式 “**数组名[下标]**” 的进一步解释：

以 “**数组名[下标]**” 的形式引用数组元素，从存储的角度，该行为的本质是：从**数组名**指示的起始位置起，访问**元素位置偏移**是**下标值**处的存储单元（该存储单元即为要访问的数组元素）。

以 `int x[3]={1,2,3};` 为例：

x数组的起始地址，设为**0xff80**



x[0]：访问从数组起始地址（即**内存地址0xff80**）开始的、**元素偏移为0**的元素。

x[1]：访问从数组起始地址开始的、**元素偏移为1**的元素。

x[2]：访问从数组起始地址开始的、**元素偏移为2**的元素。

这种解释对任何 “**数组名[下标]**” 引用元素的形式都成立，即使数组名是**形式数组**也如此。

形式数组名得到参数值以后:

如: `void bubble(int a[ ], int n)`

形参数组

形式数组名a就具有了值 **0xff80**: 代表内存中一个数组的起始地址

设, `int x[3]={1,2,3};`

调用: `bubble(x, 3);`

实参数组名

① 开辟形式数组名存储单元

0xff80

② 把x的值 (x数组的起始地址) 复值给形参数据名

x数组的起始地址, 设为0xff80



而且这个地址所指示的位置实际上就是x的起始位置

如果函数中出现了基于形式数组名的元素引用，如 **a[0]**，其含义也是：**访问从a所代表的起始地址（0xff80）开始的、下标为0的元素**（对a[1]、a[2]有同样的解释）。

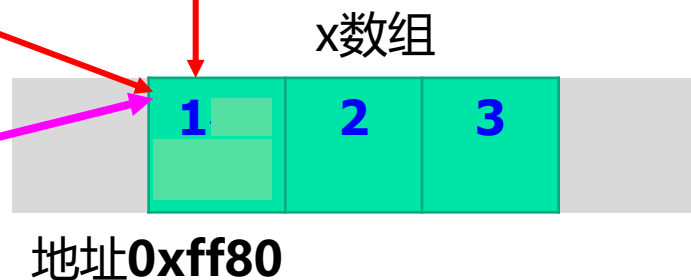
这就使得，**x[0]等价于a[0]**，它们代表同一个元素（对 x[1]和 a[1]、x[2]和a[2]有同样的解释）。此时若有：**a[0]=100;** 将导致对应元素（实参数组的元素**x[0]**）从1变为100。

如： **void bubble(int a[], int n)**

设， **int x[3]={1,2,3};**

调用： **bubble(x, 3);**

因为赋值，而使得数组中第一个元素的值从1变为100



问此时x[0]的值是多少？ 也是100

再如：

```
#include <stdio.h>
```

```
void bubble(int a[ ], int n)
```

```
{
```

```
    int i;
```

```
    for(i=0;i<n;i++)
```

```
        a[i]=a[i] * a[i];
```

```
}
```

```
int main( )
```

```
{
```

```
    int i;
```

```
    int x[3]={1,2,3};
```

```
    for(i=0; i<3; i++) printf("%d", x[i]);
```

```
    printf("\n");
```

```
    bubble(x, 3);
```

```
    for(i=0; i<3; i++) printf("%d", x[i]);
```

```
    return 0;
```

```
}
```

调用bubble前

x数组



调用bubble后



D:\CTest\555.exe

123
149

把被调函数的处理结果带回
到调用环境中——**返回结果**

简单形参也可以用地址

```
#include <stdio.h>
```

```
void change(int * px)
```

```
{  
    *px=*px+10;  
}
```

px: 指针型参数

px的存
储单元

x的地址

x

5

15

调用change前

调用change后

change中对参数的修改
结果被带回到了main函
数中——**返回结果**

```
int main( )
```

```
{  
    int i;  
    int x=5;  
    printf("x=%d\n", x);  
    change(&x);  
    printf("x=%d\n", x);  
    return 0;  
}
```

指针的内容第八章讲解

- ◆ 指针型参数也叫 **“变参”**。
- ◆ **变参**可以带回结果。

有哪些方式可以从被调函数中返回（带出）结果？

- (1) **return语句**：简单，但只能带出一个结果；
- (2) **全局变量**：程序内的耦合性大，不利于程序的开发和维护；
- (3) **变参**：可以带出多个类型不一的结果，较灵活，在有多个结果需要返回的时候经常用。

以上方式灵活使用，用好了都可以。

例 7.11 设一个整型数组 **a** 中有9个元素，元素的值大小不一。设计一个函数 **findmax**，该函数可以筛选出 **a** 中值最大的元素，并把它移到 **a** 中最后一个元素的位置（即 **a[8]**）。

算法思想：比较—交换法

因为要在所有的元素间进行比较，所以要用**一重循环**实现。

设置**循环变量 i**，**i 从 0 开始**，重复下述操作：

(I) 比较 $a[i]$ 和 $a[i+1]$ ：

- ◆ 如果 $a[i] > a[i+1]$ ，则交换二者的值；
- ◆ 否则不交换；

(II) $i++$ 。若 $i < n-1$ ，重复(I)-(II)。

- ◆ 直到 $i \geq n-1$ ，算法结束。

这个过程中，用 i 作下标来“**扫描**”数组 **a** 中的元素，故也把 i 称为数组“**扫描变量**”。

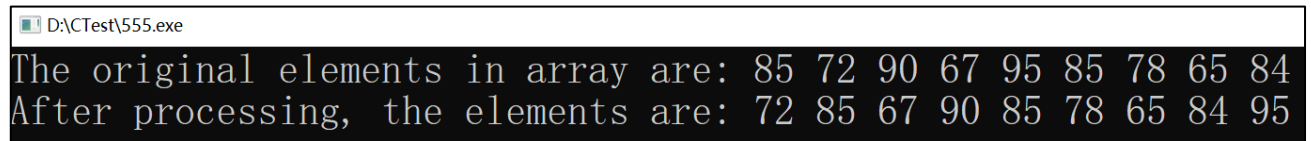
```

#include <stdio.h>
void findmax(int x[ ], int n)
{
    int t, i;
    for(i=0;i<n-1;i++)          /* i是循环控制变量, 同时也是数组的扫描变量*/
        if(x[i]>x[i+1]){        /* 如果前大后小, 则交换相邻的两个元素的值*/
            t=x[i];
            x[i]=x[i+1];
            x[i+1]=t;           }
    }
}

int main( )
{
    int k;
    int a[9]={85,72,90,67,95,85,78,65,84};
    printf("The original elements in array are: ");
    for(k=0; k<9; k++) printf("%d ", a[k]);    printf("\n");
    findmax(a, 9);
    printf("After processing, the elements are: ");
    for(k=0; k<9; k++) printf("%d ", a[k]);    /* k也用作了数组的扫描变量*/
    return 0;
}

```

} 交换 a[i] 和 a[i+1] 的值



```

D:\CTest\555.exe
The original elements in array are: 85 72 90 67 95 85 78 65 84
After processing, the elements are: 72 85 67 90 85 78 65 84 95

```

findmax 函数的执行过程中数组 a 中元素的变化情况

i	数组中元素的值								
0	85 $\Rightarrow$ 72	90	67	95	85	78	65	84	
1	72	85	90	67	95	85	78	65	84
2	72	85	90 $\Rightarrow$ 67	95	85	78	65	84	
3	72	85	67	90	95	85	78	65	84
4	72	85	67	90	95 $\Rightarrow$ 85	78	65	84	
5	72	85	67	90	85	95 $\Rightarrow$ 78	65	84	
6	72	85	67	90	85	78	95 $\Rightarrow$ 65	84	
7	72	85	67	90	85	78	65	95 $\Rightarrow$ 84	
结果	72	85	67	90	85	78	65	84	95

一维数组应用实例1——冒泡排序

1、什么是排序问题

排序是指将一个元素无序排列的序列调整成为按元素值的大小有序排列的序列的过程。

- 由小到大排列称为**升序**；
- 由大到小排列称为**降序**。

2、排序算法

对元素序列进行排序的算法称为**排序算法**。排序算法有很多，**冒泡排序算法**是其中的一种。

冒泡排序算法：

冒泡排序是一个多趟扫描的过程：

设数组 a 中有 n 个数（不失一般性，假设为整数数组）。开始的时候 a 中的元素是**无序的**。然后进入下面的**多趟扫描**过程。

双重循环：外层循环 (i) 控制趟数，内层循环 (j) 做元素扫描。

第一趟扫描($i=0$):

- (1) 置 $j=0$;
 - (2) 比较数组元素 $A[j]$ 和 $A[j+1]$ ：如果 $A[j]$ 大于 $A[j+1]$ ，
则交换二者的值，否则不变。
 - (3) $j++$ ，重复 (2) ~ (3)，直到 $j=n-2$ 时结束。
- 
- findmax
过程

第一遍扫描完成后， **a 中最大的元素被交换到数组的最后位置**

设有a数组：

a[0] a[1] a[2] a[3] a[4] a[5]

开始第一遍扫描，a[0]和a[1]比较：

31 25 12 86 42 6

交换a[0]和a[1]，然后a[1]和a[2]比较：

25 31 12 86 42 6

交换a[1]和a[2]，然后a[2]和a[3]比较：

25 12 31 86 42 6

a[2]和a[3]不交换，然后a[3]和a[4]比较：

25 12 31 86 42 6

交换a[3]和a[4]，然后a[4]和a[5]比较：

25 12 31 42 86 6

交换a[4]和a[5]比较，**第一遍扫描结束**：

25 12 31 42 6 86

无序区间

最大元素
“就位”

第二趟扫描将对前面的无序区间重复上述操作

—— 第一遍扫描把最大的元素挪到了数组的最后，就好像大元素像“气泡”一样“冒出来”，所以形象地把这种排序算法称为“**冒泡排序**”；



第2遍扫描：最大元素不再动，对**前面的n-1个数**进行第二遍比较-交换过程：做法与第1遍一样。

- ◆ 第二遍扫描后，前n-1个数中的最大元素将被挪到了整个数组倒数第二的位置上。前n-1个数中的最大元素是整个数组中的**第二大的元素**，这样次大元素也“**就位**”了。

再后，对前面n-2个尚未就位的数进行**第三遍扫描**。以此类推：

扫描	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
1	25	12	31	42	6	86
2	12	25	31	6	42	86
3	12	25	6	31	42	86
4	12	6	25	31	42	86
5	6	12	25	31	42	86

每趟一个元素就位

- 第i遍扫描后，全局的**第i大元素“就位”**。
- 直到 $i=n-1$ 时，第二小元素“就位”；
- 最后只剩下一个元素，是最小元素，处于 $a[0]$ 的位置，算法结束。


```

void bubble_sort(int a[ ], int n)  /*a 是形式数组, n 给出 a 中元素的个数*/
{
    int i, j, t, k;
    for(i=0; i<n-1; i++)          /* i 控制趟数, 共进行 n-1 趟 “冒泡” 扫描*/
    {
        for(j=0; j<n-i-1; j++)    /*j 作为扫描变量, 对当前的前 n - i 个元素进行一趟扫描*/
            if( a[j]>a[j+1] )      /* 相邻的两个元素进行比较 */
            {
                /* 若前大后小, 则交换两个元素的值 */
                t = a[j];
                a[j] = a[j+1];
                a[j+1] = t;
            }
    }
}

```

每趟扫描就是一次 findmax 过程,
冒泡排序相当于在不同的区间上执
行了 n-1 次 findmax。

```
#include "stdio.h"
```

```
void bubble_sort(int a[ ], int n); /*冒泡排序函数原型声明*/
```

```
int main(void)
```

```
{
```

```
    int k, x[ ] = {31, 25, 12, 86, 42, 6};
```

```
    bubble_sort(x, 6); /*调用冒泡排序函数，实参用数组名，并给出元素个数*/
```

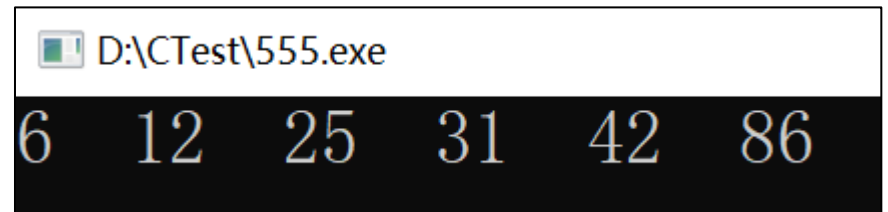
```
    for (k=0; k<6; k++)
```

```
        printf("%d ", x[k]);
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```



```
D:\CTest\555.exe
6 12 25 31 42 86
```

一维数组应用实例2：二分查找

- **查找**：在一个集合（表）中找满足条件的一个或几个元素。

找到了称为**查找成功**，找不到称为**查找失败**。

- **有序表**：表中的元素已经按照元素的值**从小到大**（或从大到小）

有序排列。反之称为**无序表**。

- **二分查找**是一个在**有序表**上的查找算法：

- **问题描述**：已知一个数 x 和一个已排好序的 n 个元素的数组 a 。

在 a 中查找 x ，如果 x 在 a 中出现，则返回 x 在 a 中的下标；
否则返回 **-1**，代表查找失败。

◆ 二分查找算法的思路：

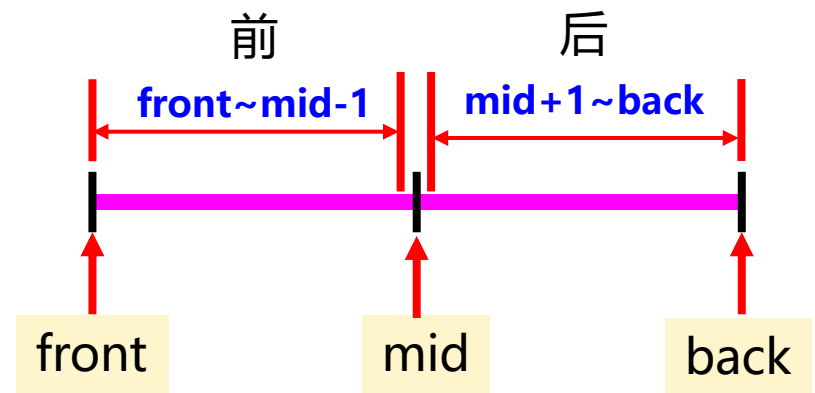
- 在下标区间 $[1..n]$ 内取中间下标 $n/2$ ，将 x 与 $a[n/2]$ 比较：
 - ▣ 如果 $x = a[n/2]$ ，则找到了 x ，返回 $n/2$ ，算法结束；否则
 - ▣ 如果 $x < a[n/2]$ ，则在数组 a 的前半部分继续查找 x ；
 - ▣ 如果 $x > a[n/2]$ ，则在数组 a 的后半部分继续查找 x ；

在 $1 \sim n/2-1$ 和 $n/2+1 \sim n$ 范围内的查找依然采用二分查找的方式——每次查找的范围“折半”，所以二分查找也叫折半查找。

```

int BinarySearch(int a[ ], int x, int n)
{
    int front=0, back=n-1, mid;
    while(front<=back) {
        mid=(front+back)/2;
        if(x<a[mid])
            back=mid-1;
        else if(x>a[mid])
            front=mid+1;
        else return (mid);
    }
    return -1;    /* 没有找到, 返回-1 */
}

```



/* 计算中间元素的下标 */

/* 查找前半部分 */

/* 查找后半部分 */

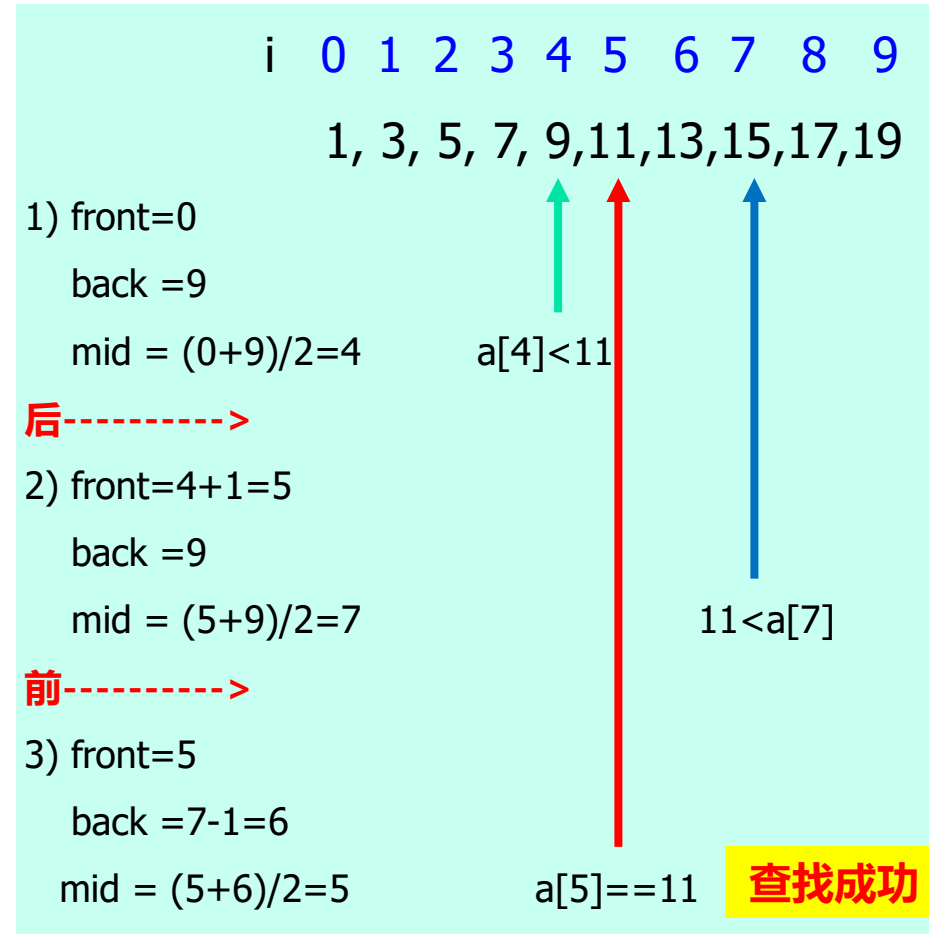
/* 找到了, 返回下标 */

```

void main(void)
{
    int x[ ]={1,3,5,7,9,11,13,15,17,19};
    int index;
    index=BinarySearch(x,11,10);
    if(index!=-1)
        printf("find %d!\n",x[index]);
    else
        printf("not find!\n");
}

```

程序的运行结果是：
find 11!



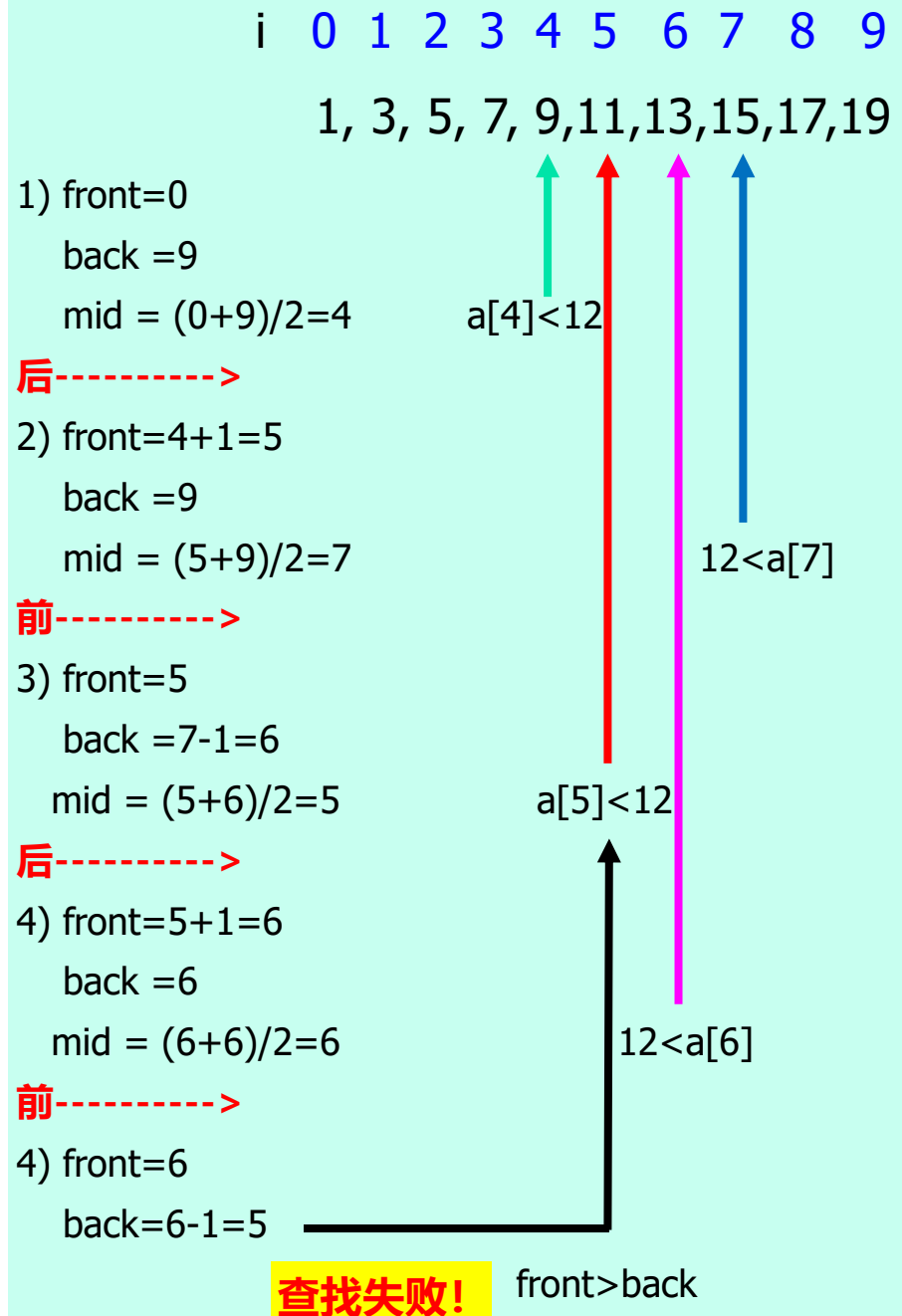
```

void main(void)
{
    int x[]={1,3,5,7,9,11,13,15,17,19};
    int index;
    index=BinarySearch(x,12,10);
    if(index!=-1)
        printf("find %d!\n",x[index]);
    else
        printf("not find!\n");
}

```

程序的运行结果是：

not find!



◆ 二分查找属于分治算法

分治策略：把大问题分解成一系列小问题，然后求解小问题，通过求解小问题来获得原始问题的解。

- 原始问题：规模最大，问题的解不明朗；
- 把原始问题一分为二，得到两个规模较小的子问题；
- 求解子问题：
 - 如果子问题的规模还比较大，继续分解，直到子问题的规模小到可以直接求解；然后进行求解。
- 原始问题的解 == 某个子问题的解（或某些子问题解的合并）

7.3 多维数组

- ◆ 一维数组中的元素之间（**位序**）呈**线性关系**。
- ◆ 二维平面上的点之间是**二维关系**。

如 **二维数据表格**（以**学生成绩表**为例）。

学号	语文	数学
01	85	91
02	82	95

- ◆ 有多行，每行一位同学的学号、语文和数学成绩
- ◆ 学号、语文成绩、数学成绩构成表格的三列。
- 其它二维数据：数学中的矩阵或行列式等。
- 怎么表示二维数据？ **用二维数组**
- ◆ **三维数据**：如三维空间里的点，需要三个坐标表述，可以定义**三维数组**描述
- ◆ **n维数组**：用n个坐标定位高维空间里的一个数据元素。

7.3.1 多维数组的声明与使用

1、多维数组的声明

类型说明 **数组名** [常量表达式1] [常量表达式2]...[常量表达式n] = {初值表};



这里**类型说明**是指：**[存储类型说明符] [类型修饰符] 数据类型**

其中，

- ◆ **数据类型**：是**指数组中基本元素的数据类型**。多维数组中的所有基本元素也都属于同一类型，所以多维数组也是同类型数据的集合。
- ◆ **n**：是数组的维数。
- ◆ **常量表达式 i**：给出第i维的大小，每个常量表达式单独用一对**[]**括起来。注：**表达式的值必须为整数**。
- ◆ 其它同一维数组。

例如： `int x[2][3];`

声明了一个二维整型数组，第一维的大小都是 2，第二维的大小都是 3。相当于**两行、三列**，共 6 个基本元素，可称为 **2×3** 的数组，数组元素的逻辑排列如：

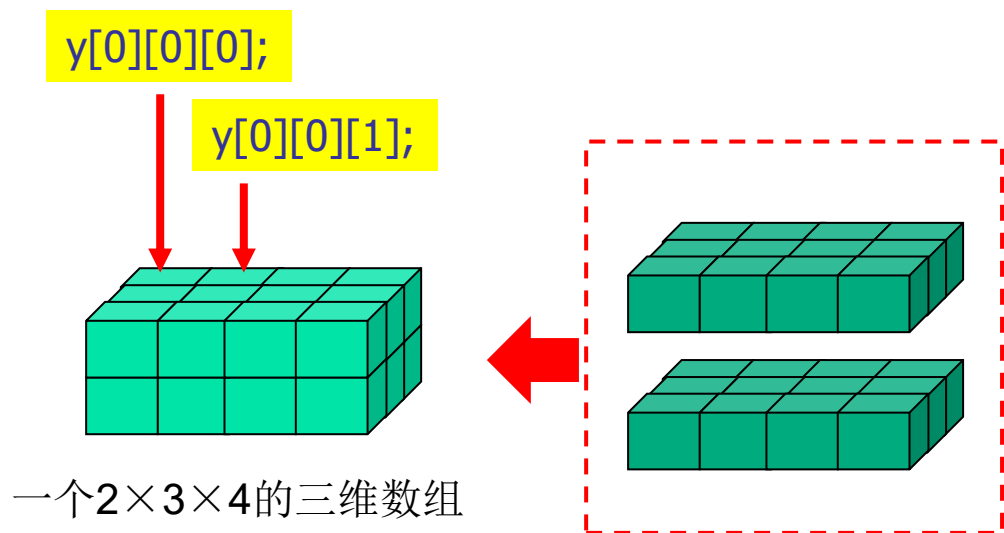
		列		
		0	1	2
行	0	x[0][0]	x[0][1]	x[0][2]
	1	x[1][0]	x[1][1]	x[1][2]

- ◆ 二维数组有两个下标：**行下标 (第一维)** 和 **列下标 (第二维)**。
- ◆ 每个元素的位置由两个下标来定位。
- ◆ **行下标和列下标都是从 0 开始**，对 N×M 的二维数组，行下标取值范围是 **0~N-1**，列下标取值范围是 **0~M-1**。

再如： **double y[2][3][4];**

声明了一个三维双精度浮点数组，它共有 $2 \times 3 \times 4 = 24$ 个元素。

逻辑结构可如图所示：



- ◆ 从空间上看，三维数组相当于**由两个二维数组堆叠而成**，每个二维数组又是 3×4 的。共有 24 个基本元素。
- ◆ 每个元素的位置由三个下标定位。每一维的下标都是从 0 开始，取值范围是 **0~本维大小-1**。

2、多维数组中元素的引用：

数组名[下标表达式1][下标表达式2]...[下标表达式n]

注：下标表达式的值必须为整数

如，二维数组 **int x[2][2];**

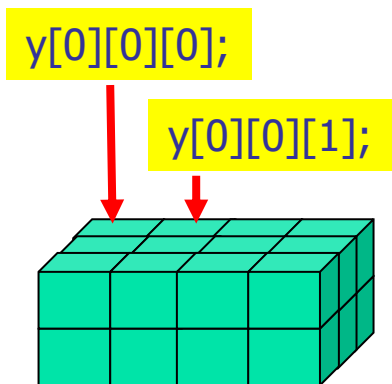
		列	
		0	1
行	0	x[0][0]	x[0][1]
	1	x[1][0]	x[1][1]

其4个元素是：

x[0][0]、 x[0][1]、 x[1][0]、 x[1][1]

↑ ↑
行下标 列下标

再如： 三维数组 **double y[2][3][4];**



其24个元素是： 层
↓ 行
↓ 列
↓ x[0][0][0]、 x[0][0][1]、
..... 、 x[1][2][3]

4维及以上的数组依次类推

例 对二维数组中元素的访问与操作示例

```
#include "stdio.h"
```

```
int main(void)
```

```
{
```

```
    int x[3][4], y[3][4]; /*定义了两个同样大小的二维数组，都是3×4的*/
```

```
    int i,j;
```

```
    for(i=0; i<3; i++){ /*i 的取值范围是0~2*/
```

```
        for(j=0; j<4; j++) /*j 的取值范围是0~3*/
```

```
            scanf("%d", &x[i][j]); /*引用 x 的元素，为 x 的元素输入数据*/
```

```
    }
```

```
    for(i=0; i<3; i++)
```

```
        for(j=0; j<4; j++) {
```

}

由于二维数组有两个下标，所以用**二重循环**处理二维数组是最典型的用法，i 和 j 用作下标。

```
        y[i][j]=x[i][j]*x[i][j]; /*引用 x 和 y 的元素进行运算*/
```

```
        printf("y[%d][%d]=%d\n", i, j, y[i][j]); /*输出 y 的元素*/
```

```
    return 0;
```

```
}
```

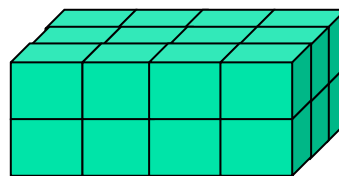
7.3.2 多维数组的存储结构

1、多维数组存储的复杂性：

数组中的元素，除了其值以外，还有位置及关系问题。如一维数组，元素 $a[i]$ 分别和 $a[i-1]$ 和 $a[i+1]$ 左、右相邻。在将一维数组存储到内存中一块连续字节构成的线性空间时，这种位置关系得到了“自然”保持。

但对高维数组，除了数据本身，数据间也有位置关系，但比较复杂，如二维数组中的元素间有上、下、左、右四个方向的相邻关系，更高维的数组中元素间的逻辑位置关系就更复杂。

$x[0][0]$	$x[0][1]$
$x[1][0]$	$x[1][1]$



存储数据的基本要求：不仅能保存数据的值，还要能定位，以便要访问的时候能方便地找到它们；还要把数据间的关系保存起来，需要的时候能够把它们邻居或其它关联数据“找”出来。

如何保存数据间的关系？

通过一定的“关联”“体现”出来。

如，一维数组通过存储单元的物理位置相邻“体现”出元素间左右位置逻辑相邻，从而可由 $a[i]$ 方便地找到它的两个邻居。

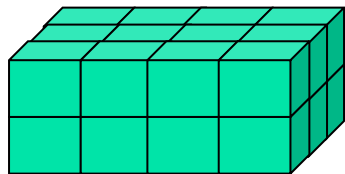
但对二维、三维等高维数组，元素间有更多的逻辑位置关系，而不止线性，但内存字节空间是线性的，高维数组元素之间的复杂逻辑位置关系又该如何表示出来呢？

2、多维数组的存储

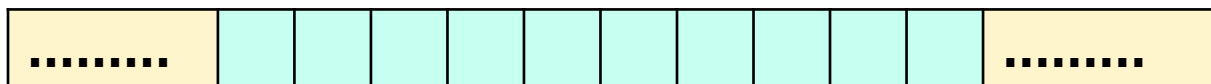
首先不管是一维数组还是多维数组，存储时，系统都是为其分配一块**连续的字节空间**保存数组元素的。但**问题是**：

字节空间是**线性编址**的，字节间只有一维的位置邻接关系。但**高维数组的元素间具有多维逻辑位置结构**。那么如何在保存数组元素个体的同时，也将其复杂的逻辑位置关系保存起来呢？

x[0][0]	x[0][1]
x[1][0]	x[1][1]



内存空间的字节间仅有一维的线性位置关系



如何映射？

高维数组的元素间具有高维逻辑位置关系

3、 “行主序存储” 的映射机制：

(1) 首先系统为数组分配一块**连续的字节空间**，若数组中的基本元素总共有N个，则

分配的字节总数 = $N * \text{sizeof}(\text{数组类型})$

如，`int x[3][4]`，共12个int型元素，所以总共分配48个字节。

(2) 然后将这些字节**连续分成N段**，每段对应一个基本的**元素存储单元**。之后采用 “**行主序**” 的方式将数组中的元素依次映射到存储单元中，这样既存储了元素的值，也方便了元素的定位和确定元素间的逻辑位置关系。

(1) 二维数组的**行主序**存储

设有二维数组 x: `int x[2][3]={85, 91, 88, 82, 95, 87};`

- 二维数组 x 的**逻辑结构**:

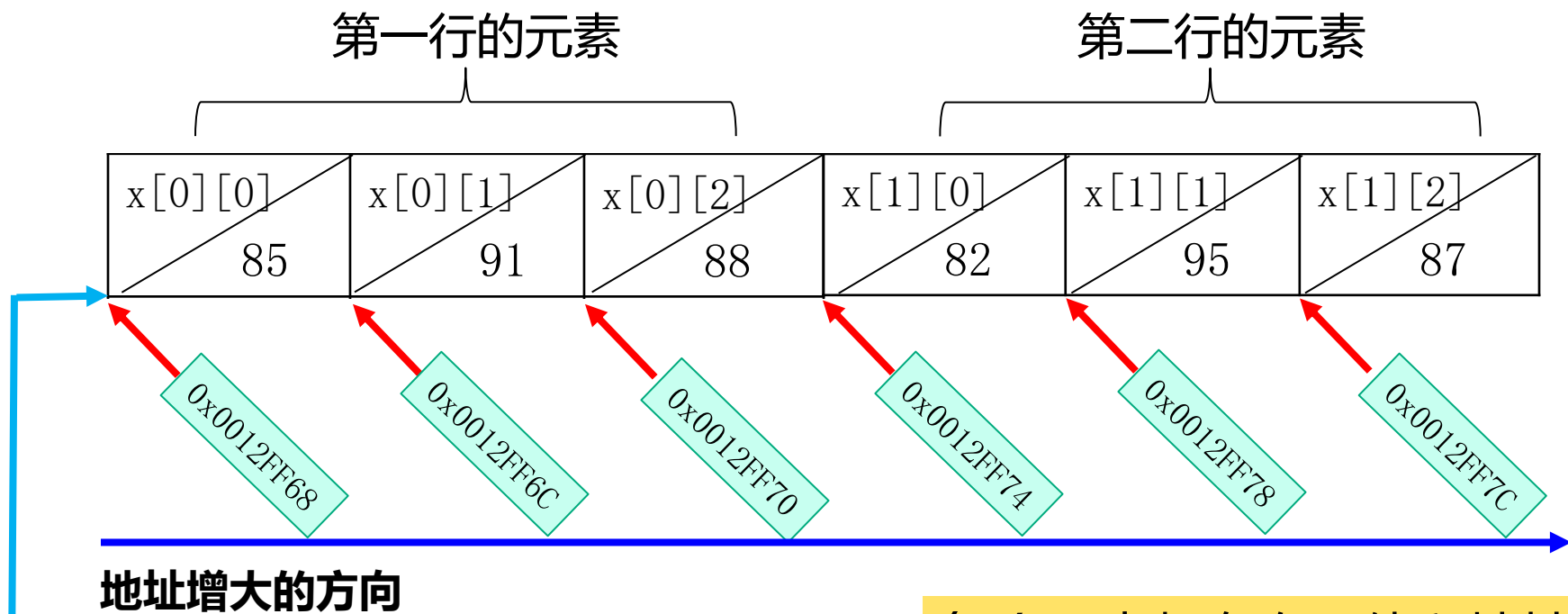
行	列		
	$x[0][0]$ 85	$x[0][1]$ 91	$x[0][2]$ 88
	$x[1][0]$ 82	$x[1][1]$ 95	$x[1][2]$ 87

- 二维数组x的**行主序存储结构**

$x[0][0]$ 85	$x[0][1]$ 91	$x[0][2]$ 88	$x[1][0]$ 82	$x[1][1]$ 95	$x[1][2]$ 87
第一行的元素			第二行的元素		

行主序排列: 先存第一行的元素, 接着再存第二行的元素, ...。

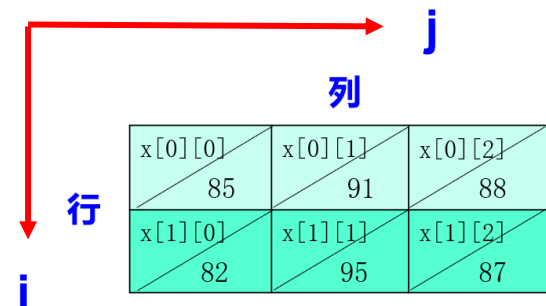
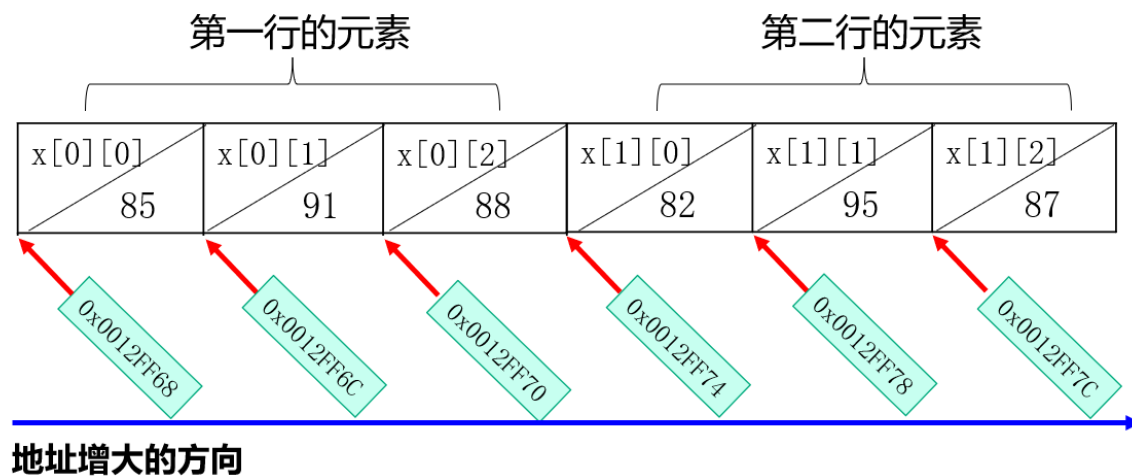
数组和元素的地址



- ◆ 数组的存储空间的一个字节的地址既是第一个基本元素的地址，也是整个数组的起始地址(**数组的地址**)。
- ◆ 但对高维数组而言，二者的**类型**(指地址类型)和**指代的对象**是完全不同的。

每个元素都有自己值和地址

地 址	元 素	值
0x0012FF68	$x[0][0]$	85
0x0012FF6C	$x[0][1]$	91
0x0012FF70	$x[0][2]$	88
0x0012FF74	$x[1][0]$	82
0x0012FF78	$x[1][1]$	95
0x0012FF7C	$x[1][2]$	87



二维数组元素位置的一般换算关系： 设二维数组是 **T a[N][M]**

元素 **a[i][j]** 相对数组的**起始位置**，在内存中存储的 **(元素单元) 位置偏移** 是： **$i * M + j$** 。所以

$$a[i][j] = ((T*)a)[i * M + j]$$

而其上、下、左、右的邻居是：

上： $((T*)a)[(i-1)*M+j]$

下： $((T*)a)[(i+1)*M+j]$

同列

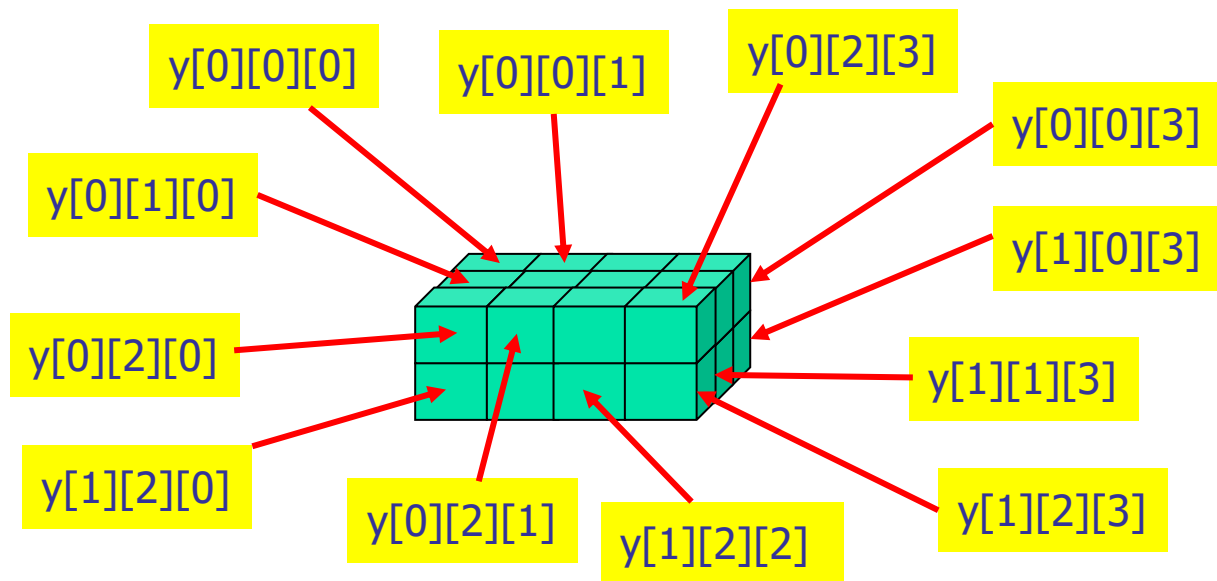
左： $((T*)a)[i*M+j-1]$

右： $((T*)a)[i*M+j+1]$

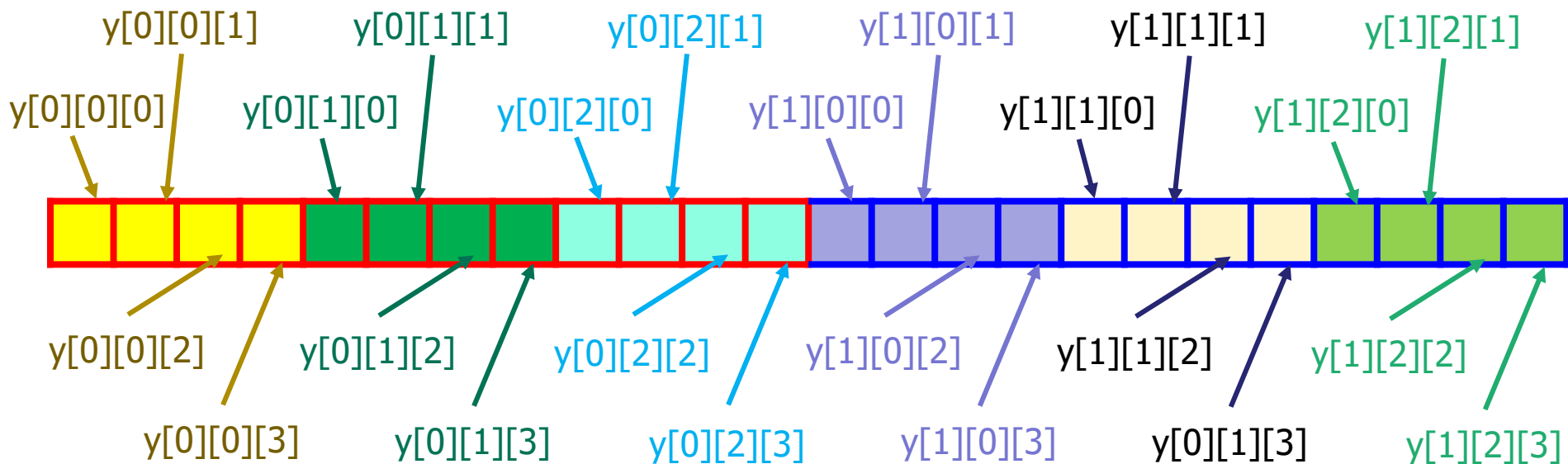
同行

(2) 三维数组的行主序存储 设有三维数组x: `int y[2][3][4];`

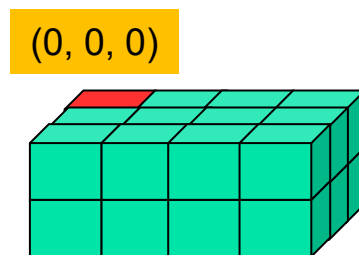
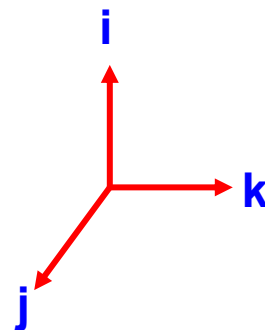
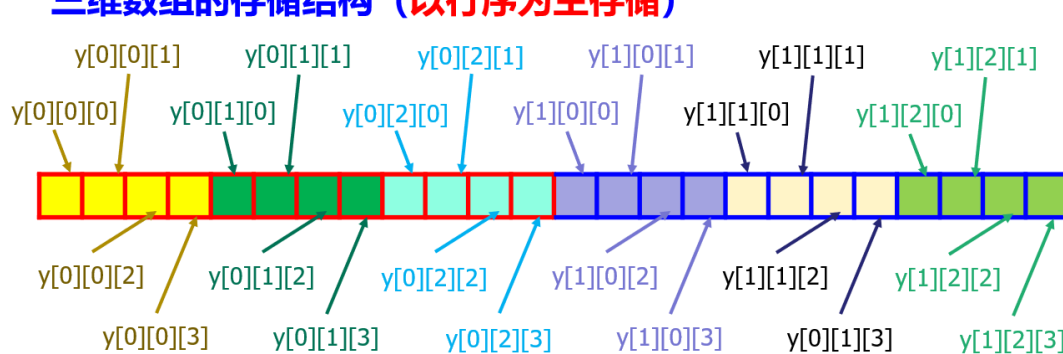
三维数组的逻辑结构



三维数组的存储结构 (以行序为主存储)



三维数组的存储结构 (以行序为主存储)



三维数组元素位置的一般换算关系：设三维数组是 $T a[N][M][K]$

元素 $a[i][j][k]$ 相对数组的起始位置，在内存中存储的 (元素单元) 位置偏移是： $i * M * K + j * K + k$ 。所以

$$a[i][j][k] = ((T^*)a)[i * M * K + j * K + k]$$

其左、右、前、后、上、下相邻的元素是：

左： $((T^*)a)[i * M * K + j * K + k - 1]$

右： $((T^*)a)[i * M * K + j * K + k + 1]$

前： $((T^*)a)[i * M * K + (j + 1) * K + k]$

后： $((T^*)a)[i * M * K + (j - 1) * K + k]$

上： $((T^*)a)[(i - 1) * M * K + j * K + k]$

下： $((T^*)a)[(i + 1) * M * K + j * K + k]$

同层

层间

2、数组和元素的地址

◆ 数组的地址：

- **数组名**就代表数组的地址，其值是数组的起始地址值，是**地址常量**，不能被修改；不能再用**&**取“数组名的地址”。

◆ 数组元素的地址：

- 每个元素都有地址，用 **&数组名[下标]** 取元素的地址。

地 址	元 素	值
0x0012FF68	x[0][0]	85
0x0012FF6C	x[0][1]	91
0x0012FF70	x[0][2]	88
0x0012FF74	x[1][0]	82
0x0012FF78	x[1][1]	95
0x0012FF7C	x[1][2]	87

int a=x[0][0] → a等于85

&x[0][0] → 0x0012FF68

scanf("%d", &x[i][j]);

注：&x[i][j] 等价于 **&(x[i][j])**

运算表达式中，“**数组[下标]**”要当作一个整体看待，等价于一个变量。

7.3.3 多维数组的初始化

也是用**初值表** (`= {...}`) 给多维数组进行初始化。由于有**高维结构**，所以多维数组初始化有两种形式：

(1) 按照**存储结构**进行初始化

此时所有元素的初值写在“**一重**”**花括号**中，**按数组元素的内存存储顺序**依次为每个元素赋初值。

如： `int x[2][3]={85, 91, 88, 82, 95, 87};`

<code>x[0][0]</code>	<code>x[0][1]</code>	<code>x[0][2]</code>	<code>x[1][0]</code>	<code>x[1][1]</code>	<code>x[1][2]</code>
85	91	88	82	95	87

- ◆ 初值表中可以缺省初值或为空，但只能缺省后面的元素。
- ◆ 没有初值对应的元素的初值，编译器自动置0。

如： `int x[2][3]={85, 91, 88, 82};` 或 **`int x[2][3]={ };`** 79

(2) 按照**逻辑结构**进行初始化:

此时用**嵌套**的“**多重**”**花括号**给出数组内部的高维组织结构，按逻辑结构的关系依次为各元素赋初值。如：

```
int x[2][3]={ { 85,91,88 } , { 82,95,87 } };
```

```
int d[2][3][4]={ { { 1, 2, 3, 4}, { 5, 6, 7, 8} , { 9,10,11,12}},  
                 { {13,14,15,16},{17,18,19,20} , {21,22,23,24}} };
```

- 可读性好，但初值表的形式与数组的维数有关；
- 初值表中**各重花括号**中都可以有缺省初值或为空，但都只能缺省后面的元素。
- 没有初值对应的元素，编译器自动置0.

指出下面数组定义的对与错：

int x[2][3]={85, , , 82, 95, 87}; ❌

int y[2][3]={**{85,91}**, **{82,95,87}**}; ✔️

int z[2][3][4]={**{{1, 2},{5, 6, 7, 8}}**, **{{13},{ },{21}}**}; ✔️

int z[2][3][4]={**{{1, 2},{5, 6, 7, 8}}**, **{{13}, ,{21}}**}; ❌

int s[2][3][4]={**{{1, 2}, ,{9,10,11,12}}**, **{{13},{ },{21}}**}; ❌

int t[2][3][4]={**{{ }}**}; ✔️

任何情况下，只能缺省后面的初值。

当初值表中列出了所有数组元素的初值时，数组声明中的

第1维大小的说明可以省略，但其它维的大小不能省略。此时，

编译器可根据初值个数推算出第一维的大小。

如：int x[][3]={ { 85,91,0}, {82,95,0}};

int d[][2][2]={ {{1,2},{3,4}},{{5,6},{7,8}}};

只能省略第一维的大小！

再如：int x[2][]={ { 85,91,0}, {82,95,0}}; ❌

int d[][][2]={ {{1,2},{3,4}},{{5,6},{7,8}}}; ❌

7.3.4 多维数组作为函数参数

1、在函数参数表中将形参数组声明为多维数组形式

如：

```
void func(int a[ ][3], int n)
```

```
void func2(int x[ ][3], int y[ ][3][4], int n1, int n2)
```

关于形参数组的声明：

- ◆ 除了第一维外，**其余各维必须指定大小**。
- ◆ 第一维不用指定大小（**指定了也没有实际意义**），另有整型参数（如 int n 或 int n1, int n2）给出形式数组第一维的大小。

2、函数调用

调用函数时，先在主调函数中声明与被调函数的形式数组维数大小一致（除了第一维）的实参数组；然后调用函数，**实参数表中直接用实参数组名，并至少指出实参数组第一维的大小。**

如：

```
void func(int a[ ][3], int n)
```

```
void func2(int x[ ][3], int y[ ][3][4], int n1, int n2)
```

设 `int p[2][3];`

`int q[5][3][4];`

调用： `func(p, 2);`

或： `func2(p, q, 2, 5);`

额外的整数给出实参多维
数组第一维的大小

数组实参直接用数组名

3、参数传递

与一维数组做参数时的参数传递机制完全一样，多维数组作参数时，也是把实参数组的起始地址（**实参数组名的值**）复制给对应的形参数组名，从而形参数组名就具有和实参数组名一样的值，**均是实参数组的起始地址**。

设 `int p[2][3];`

```
void func(int a[ ][3], int n)
```

`int q[5][3][4];`

```
void func2(int x[ ][3], int y[ ][3][4], int n1, int n2)
```

调用： `func(p, 2);`

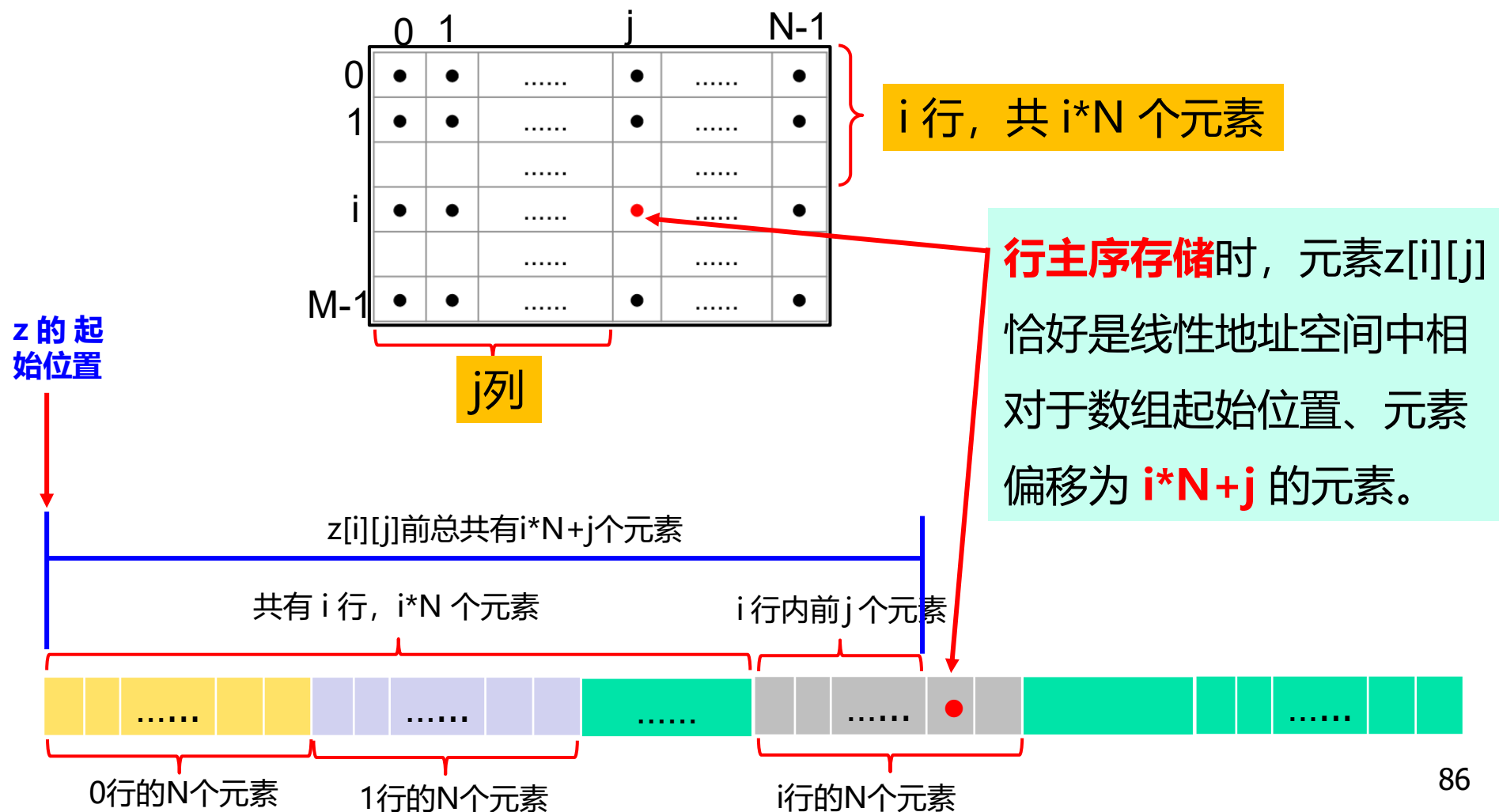
或： `func2(p, q, 2, 5);`

参数传递后，p的值复制给被调函数中的形参数组名a，a的值也就等于实参数组 p 的起始地址。

参数传递后，被调函数中的形参数组名 x 的值是实参数组 p 的起始地址，形参数组名 y 的值是实参数组 q 的起始地址。

4、元素的引用

以二维数组 `int z[M][N]` 为例，`z[i][j]` 的含义是：**访问相对于数组的内存起始位置、元素偏移为 $i*N+j$ 的元素。**

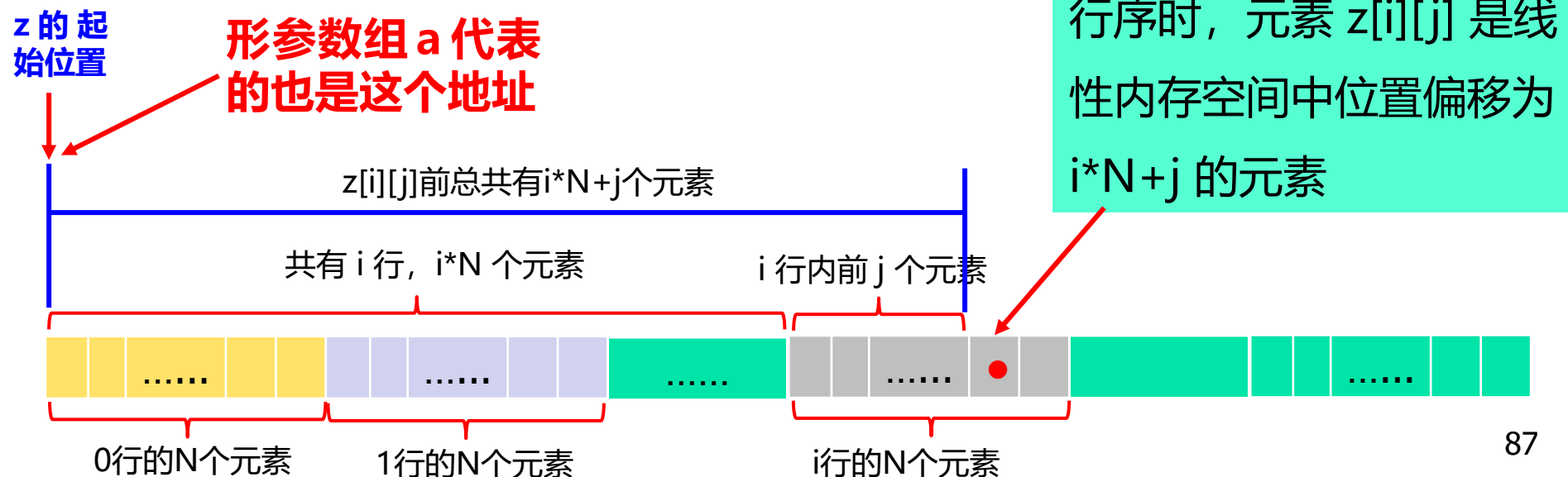


5、基于形参数组的元素引用

```
void f(int a[ ][N], int n)
```

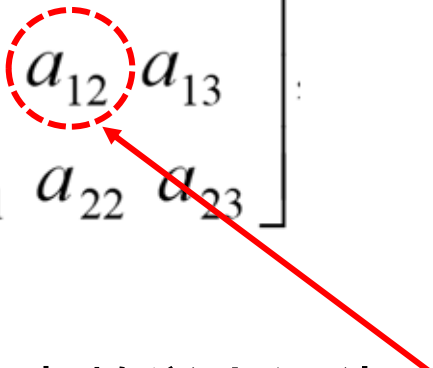
以二维形参数组 `int a[ ][N]` 为例：

实参和形参结合后，`a` 的值就是实参数组的起始地址，基于形参数组 `a` 访问元素 `a[i][j]` 的含义与基于实参数组 `z` 访问元素 `z[i][j]` 的含义相同，即：**访问相对于起始位置、元素偏移为 $i*N+j$ 的元素**。而此时 `a` 的值就是实参数组 `z` 的起始地址，所以 `a[i][j]` 实际上就是 `z[i][j]`（和一维数组的规则是一样的）。



二维数组的应用1——矩阵相加

矩阵：如下，以二维矩形“行-列”形式排列的一组数称为矩阵。

$$A = \begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \end{bmatrix}$$


设矩阵有n行、m列，则称该矩阵为 **$n \times m$ 的矩阵**，记为 **$A_{n \times m}$** ，并称**矩阵的规模是 $n \times m$** 的。

- ◆ 矩阵中的元素数学上记为 **a_{ij}** ，其中i是行坐标，j是列坐标。
- ◆ 矩阵中的元素呈“行-列”排列，是二维结构，程序里就用二维数组存储矩阵数据。
- ◆ 二维数组中i行j列的元素是 $A[i][j]$ 。
- ◆ 常见的矩阵运算是**矩阵加**和**矩阵乘**。

矩阵的加法

设有两个 $n \times m$ 的矩阵 A 和 B , $A+B$ 的结果也是一个 $n \times m$ 的矩阵, 记为 C , 即: $\mathbf{C}_{n \times m} = \mathbf{A}_{n \times m} + \mathbf{B}_{n \times m}$

- ◆ C 中的元素按照以下规则计算: $c_{ij} = a_{ij} + b_{ij}$
- ◆ 做矩阵加法运算的两个矩阵的维数必须一样, 即都是 $n \times m$ 的; 且结果矩阵 C 也是 $n \times m$ 的。

用程序实现两个矩阵相加, 程序的结构是一个双重循环:

按行-列的顺序依次完成矩阵 A 和 B 所对应的二维数组中的元素相加, 并将结果保存在结果矩阵 C 所对应的二维数组中。

```
void add_matrix(int a[ ][M], int b[ ][M], int c[ ][M], int n, int m)
```

```
{
```

```
    int i, j;
```

```
    for(i=0; i<n; i++)
```

```
        for(j=0; j<m; j++){
```

```
            c[i][j] = a[i][j]+b[i][j];
```

```
        }
```

```
}
```

A、B、C三个二维数组都是
n×m大小，其中，**m等于M**。

“对应元素” 相加

存储顺序对访存效率的影响： 行主序存储时，按行处理效率高；
列主序存储时，按列处理效率高；

```

#include "stdio.h"

#define N 3
#define M 4

void add_matrix(int a[ ][M], int b[ ][M], int C[ ][M], int n, int m);

int main(void)
{
    int A[N][M]={ {1, 2, 3, 4},{5, 6, 7, 8},{9, 10, 11, 12}};
    int B[N][M]={11,12,13,14,15,16,17,18,19,20,21,22};
    int C[N][M], i,j;
    add_matrix(A, B, C, N, M);
    for(i=0;i<N;i++){
        for(j=0;j<M;j++) printf("%8d",C[i][j]);
        printf("\n");
    }
    return 0;
}

```

程序的运行结果为：

12	14	16	18
20	22	24	26
28	30	32	34

二维数组的应用2——矩阵乘法

设有两个矩阵 $A_{n \times k}$ 和 $B_{k \times m}$ ，则矩阵 A 乘以矩阵 B 的结果为一个 $n \times m$ 的矩阵 $C_{n \times m}$ ，记为 $C_{n \times m} = A_{n \times k} \times B_{k \times m}$

其中，C 中的元素 c_{ij} 按照以下规则计算：

$$c_{ij} = \sum_{p=1}^k a_{ip} b_{pj}$$

A的i行与B的j列的**内积**

即： $c_{ij} = a_{i1} * b_{1j} + a_{i2} * b_{2j} + \cdots + a_{ik} * b_{kj}$

$$1 \leq i \leq n, 1 \leq j \leq m$$

如:

$$A = \begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \end{bmatrix}, \quad B = \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \\ b_{20} & b_{21} & b_{22} \\ b_{30} & b_{31} & b_{32} \end{bmatrix}$$

$$C = A \times B, \quad \text{其中 } c_{ij} = \sum_{k=0}^3 a_{ik} b_{kj} \quad C = \begin{bmatrix} c_{00} & c_{01} & c_{02} \\ c_{10} & c_{11} & c_{12} \\ c_{20} & c_{21} & c_{22} \end{bmatrix}$$

$$\text{其中, } c_{00} = a_{00} * b_{00} + a_{01} * b_{10} + a_{02} * b_{20} + a_{03} * b_{30}$$

$$c_{01} = a_{00} * b_{01} + a_{01} * b_{11} + a_{02} * b_{21} + a_{03} * b_{31}$$

$$c_{02} = a_{00} * b_{02} + a_{01} * b_{12} + a_{02} * b_{22} + a_{03} * b_{32}$$

$$c_{10} = a_{10} * b_{00} + a_{11} * b_{10} + a_{12} * b_{20} + a_{13} * b_{30}$$

.....

内积

矩阵乘要求：相乘的两个矩阵，第一个矩阵的第二维和第二个矩阵的第一维大小必须相同，即**第一个矩阵的列数和第二个矩阵的行数必须相同。**

如：

$$\begin{matrix} 3 \times 4 & A = \begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \end{bmatrix}, & B = \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \\ b_{20} & b_{21} & b_{22} \\ b_{30} & b_{31} & b_{32} \end{bmatrix} & 4 \times 3 \end{matrix}$$

$$\begin{matrix} 3 \times 3 \\ C = A \times B, \quad \text{其中 } c_{ij} = \sum_{k=0}^3 a_{ik} b_{kj} \end{matrix}$$

用程序实现矩阵相乘

(1) 矩阵用二维数组表示, 如: `int A[N][K], B[K][M], C[N][M];`

(2) 程序用**三重循环**实现:

- ◆ 最外层 i 循环用于控制行: $i=0\sim n-1$
- ◆ 中间的 j 循环用于控制列: $j=0\sim m-1$
- ◆ 最内层 p 循环用于计算内积: $p=0\sim k-1$

```
void mul_matrix(int a[ ][K], int b[ ][M], int c[ ][M], int n, int k, int m)
```

```
{
```

```
    int i, j, p, sum;
```

```
    for(i=0; i<n; i++)
```

```
        for(j=0; j<m; j++){
```

```
            sum=0;
```

```
            for(p=0; p<k; p++)
```

```
                sum+=a[i][p]*b[p][j];
```

```
            c[i][j]=sum;
```

```
        }
```

```
    }
```

$$A = \begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \end{bmatrix}, \quad B = \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \\ b_{20} & b_{21} & b_{22} \\ b_{30} & b_{31} & b_{32} \end{bmatrix}$$

$$C = A \times B, \quad \text{其中 } c_{ij} = \sum_{k=0}^3 a_{ik} b_{kj}$$

内积： “乘” 并 “求和”

```

#include "stdio.h"
#define N 3
#define K 4
#define M 3
void mul_matrix(int a[ ][K],int b[ ][M],int C[ ][M],int n,int k,int m);
int main(void)
{
    int A[N][K]={{1,2,3,4},{5,6,7,8},{9,0,1,2}};
    int B[K][M]={{1,2,3},{4,5,6},{7,8,9},{0,1,2}};
    int C[N][M], i, j;
    mul_matrix(A, B, C, N, K, M);
    for(i=0; i<N; i++){
        for(j=0; j<M; j++) printf("%8d ", C[i][j]);
        printf("\n");
    }
    return 0;
}

```

程序的运行结果为:

30	40	50
78	104	130
16	28	40

7.3.4 用“数组类型”和“超级数组”的观点看待高维数组

C语言的基本数据类型中没有“**数组类型**”的概念，但可以这样看待数组：

事实上可以自定义，以后讲解

(1) 一维数组，以 `int a[3]` 为例：

将 `int [3]` 视为一种**一维数组类型**，即“由三个int元素组成的一维数组”类型（可看作是基本int类型的一种扩展），则

`int a[3];`

就相当于声明了一个属于 `int [3]` 类型的变量 `a`。

—— `a` 就是这样一个属于**一维数组类型**的“**超级简单变量**”。

(2) 二维数组，以 `int b[2][3]` 为例：

同样将 “`int [3]`” 看作是一个 “由3个int型元素组成的一维数组类型”，则 `int b[2][3]` 相当于声明了一个元素是 `int [3]` 类型的一维数组：

- ◆ 数组名为b、数组的大小为2，而这个b的元素是 “`int [3]` 类型的一维数组”：

- `b[0]`是第一个 “`int [3]` 型的一维数组”，`b[0]`可视为该数组的名字；
- `b[1]`是第二个 “`int [3]` 型的一维数组”，`b[1]`可视为该数组的名字；。

那么b是什么？ `b`是由两个一维数组组成的一维数组
—— 超级一维数组。

(3) 三维数组，以 `int c[2][3][4];` 为例：

将 `int [3][4]` 看作是一种 “**3x4的int型二维数组类型**”，则 `c` 可以看作是由两个这种**二维数组类型**的元素组成的 “**一维数组**”。

- ◆ `c[0]` 表示第一个二维数组，可视为该二维数组的名字；
 - ◆ `c[1]` 表示第二个二维数组，可视为该二维数组的名字。
- `c` 就是由两个这样的二维数组组成的**超级一维数组**。

以此类推，一般情况下，一个 **n 维数组** 可以看作是以 **n-1 维数组** 为元素的 “**超级**” **一维数组**。

三维数组还可以这样看，同样以 `int c[2][3][4];` 为例：

将 “`int [4]`” 看作是一种 “由4个int型元素组成的一维数组类型”，则 `c` 可以看作是由 2×3 ，共6个这种一维数组类型的元素组成的 “二维数组”：

- ◆ 这个二维数组的大小是 2×3 ，元素是长度为4的整型一维数组，共有6个。
- ◆ `c[0][0]` 表示第一个一维数组，`c[0][1]` 表示第二个一维数组，……，`c[1][2]` 表示第六个一维数组。

—— `c` 就是由六个这样的一维数组组成的超级二维数组。

其他情况可以依此类推。理解这些对后面学习如何定义 “数组类型” 及 “指向数组” 的指针有重要意义。

7.3.5 用 `typedef` “定义” 类型

1、类型表达式

C语言中的**表达式**分成两类：

如 “第二章：把由**运算符**和**操作数**组成的式子称为**表达式**。

如： $x+y$

这一类表达式称为**值表达式**（或**运算表达式**），目的是运算并得出一个结果。

还有一类表达式叫**类型表达式**。

类型表达式：是由**类型说明符**和**数据类型名**组成的式子，**目的是用于说明一种数据类型。**

其中，

◆ **类型说明符：有三个**

- **()**：用于将一个标识符说明为**函数**，或改变解释的**优先级**；
- **[]**：用于将一个标识符说明为**数组**；
- *****：用于将一个标识符说明为**指针**。

■ **数据类型名：分为两类**

- 一类是**基本数据类型名**，如**int**、**char**、**double**等。
- 另一类是**构造类型名**，如**枚举**、**结构**、**联合**等自定义类型。

在解释**类型表达式**时，对**类型说明符**和**数据类型名**要按照下列**优先级与结合性**的次序依次解释。

级别	符号	优先级	结合性
1	()、[]	最高	从左至右
2	*	次之	
3	数据类型名	最低	

程序中**类型表达式**可以出现在两个地方，

(1) 在一般的**变量声明语句**中出现：

如： `int *p[3];`

↑
去掉变量名，剩余的 `int * [3]` 就是一个**类型表达式**。

- ◆ **变量声明语句的作用**：将一个标识符声明为**某种类型的变量**。

如上，将p声明为**int * [3]**类型的变量。

- ◆ **如何解释类型表达式int * [3]呢？**

按照**类型说明符**和**数据类型名**的**优先级**与**结合性**，有：

[3] → * → int

即解释为：

含有三个元素的**数组类型** → **指针数组类型** → **整型指针数组类型**

而对变量定义语句：**int *p[3]** 语句的解释为：

类型表达式 **int * [3]** 作用于标识符 **p**，**p** 与类型说明符和
数据类型名按优先级、结合性依次发生作用，从而有：

$$\begin{array}{ccccccc} \mathbf{p} & \rightarrow & \mathbf{[3]} & \rightarrow & \mathbf{*} & \rightarrow & \mathbf{int} \\ & & \textcircled{1} & & \textcircled{2} & & \textcircled{3} \end{array}$$

- 由①得：**p**是一个由3个元素组成的**数组**；
- 由②得：**p**是一个有3个元素的**指针数组**；
- 由③得：**p**是一个有3个元素的**int类型的指针数组**。

(2) 在**typedef**定义语句中出现

2、typedef 类型定义语句

typedef: C语言的一个关键字（命令）

作用是**用一个标识符为一个类型表达式命名**。

用typedef为类型表达式命名有两种形式：

(1) 为已有的“简单”数据类型起“**别名**”，形式为：

typedef 已有数据类型名 标识符表列；

简单的类型表达式

其中，

- ◆ **标识符表列**：1个或多个标识符的列表，多个时用逗号隔开。
- ◆ **功能**：将标识符表列中的每个标识符定义为已有数据类型的**等价表示**，即给已有数据类型起1个或多个**别名**。

例如： **typedef unsigned int UINT, size_t;**

就是将**UINT**和**size_t**都定义成为**unsigned int**类型的别称。之后可以用 **UINT**或**size_t**声明变量，与用**unsigned int**声明变量等价，如：

UINT x;

或： **size_t y;**

(2) 为复杂类型表达式 “命名”

如： `typedef int A4 [4];`

`int [4]`：类型表达式，表示一种有4个元素的一维整型数组类型；

`A4`：用 `typedef` 命令为类型表达式 `int [4]` 命的名，也相当于为类型表达式 `int [4]` 起了个 “别名”。

之后就可以用 “类型名” `A4` 来声明属于这种类型的变量。

如： `A4 a4` ：声明 `a4` 是 `A4` 类型的变量。

相当于 `int a4[4];`（即一个有4个元素的一维整型数组）

`A4 a54[5]` 是什么含义？ `int a54[5][4]`

注意写法特点：标识符写在类型表达式中间。

再如：设有 `typedef int A34 [3][4]`

`A34 a34` 是什么含义？

```
int a34[3][4];
```

`A34 a534[5]` 是什么含义？

```
int a534[5][3][4]
```


(3) 更复杂的类型表达式

如：typedef char *(*p_to_fun)(char *,char *);

其中，

◆ **粉色 部分：** char *(*)(char *,char *) 是**类型表达式**，

说明了一种 “**函数指针类型**” ；

◆ **p_to_fun**：用于为该类型表达式命名的标识符。

—— 这种含**标识符**和**类型表达式**的式子称为**类型定义表达式**。

注： char *(*)(char *,char *) **代表的数据类型描述：**一种以两个字符指针作为形参、返回值类型也为字符指针的函数指针类型。

3、typedef 类型 “定义” 语句

用 typedef “命名” 类型的一般语句形式是：

typedef 类型定义表达式;

功能：用类型定义表达式中的**标识符**为类型表达式**命名**。

如：**typedef char *(*p_to_fun)(char *,char *);**

即：用标识符**p_to_fun**为类型表达式 **char *(*)(char *,char *)** 命名。

然后 **p_to_fun** 就可作为 “**类型名**”，用以代表**类型表达式**
声明 “**该类型**” 的变量。如：**p_to_fun ptof;**

可称呼： **ptof**是一个**p_to_fun**类型的变量。

思考：如果不用上述“定义”的类型 **p_to_fun**，该如何声明一个 **char *(*)(char *,char *)** 类型的函数指针变量ptof？

```
char *(* ptof)(char *, char *);
```

该方式与前面用 **p_to_fun ptof**; 声明等价。

令： **ptof = strstr**;

注：C的库函数**strstr**的函数原型是
`char *strstr(char *s,char *t);`

设： **char s1[] = "University", s2[] = "sit"**;

可用表达式 **ptof(s1,s2);** 或 **(*ptof)(s1,s2);** 调用**strstr**函数

7.5 字符数组和字符串

7.5.1 字符数组

以**字符型数据**为元素的数组就是**字符数组**。

- ◆ 字符数组的元素类型：**char**（或wchar_t）；

如：**char** s[81];

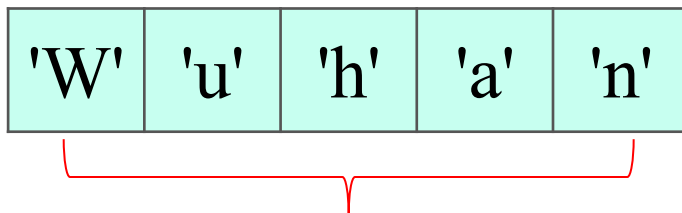
char t[20][30]; 等

- ◆ 字符数组元素的引用：**数据名+下标**

如：s[1] 或 t[0][0]

- **字符数组的存储**：一个char型字符数据在内存中占一个字节，所以一个**长度为n的一维字符数组**在内存中占据**n个连续字节空间**存储其元素。

如 char s[]={'W', 'u', 'h', 'a', 'n'};



自动计算数组大小，分配5个字节存储5个字符

理论上，**字符数组仅是元素是字符型数据的数组**，因此除了元素类型不同以外，其它属性，如**数组的声明、初始化、下标取值、元素引用、数组的地址、元素的地址、高维数组的定义与使用**等均与其它类型的数组相同。

7.5.2 字符串

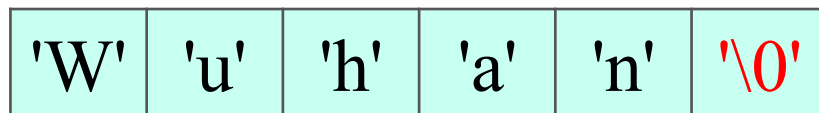
用一对**双引号**界定的一个字符序列称为**字符串**。

如： `"Huangzhong University of Science and Technology"`

1、字符数组和字符串的关系

C语言没有特定的字符串类型，而是仅**用字符数组来存储字符串**：字符串中的字符在字符数组中顺序存放，并在最后一个字符后加**空字符'\0'**表示结束。

如 `"Wuhan"` 在内存中的存储：



书写时字符串最后不用显式加'\0'，存储时编译器会自动加上。

这个0是字符数组的一部分，不可或缺

2、字符串的长度

字符串的长度 = 字符串中 “实际” 的字符数，不包括'\0'。

如：

'W'	'u'	'h'	'a'	'n'	'\0'
-----	-----	-----	-----	-----	------

字符串长度为5

注：**字符串的长度不计最后的'\0'。**

3、字符串的存储长度

字符串的存储长度是指为了存储字符串而**占用的字节数**。

字符串存储长度=字符串的长度+1

所以，用于存放字符串的字符数组的长度至少应等于该字符串的长度+1。否则会发生溢出。

4、字符数组的初始化

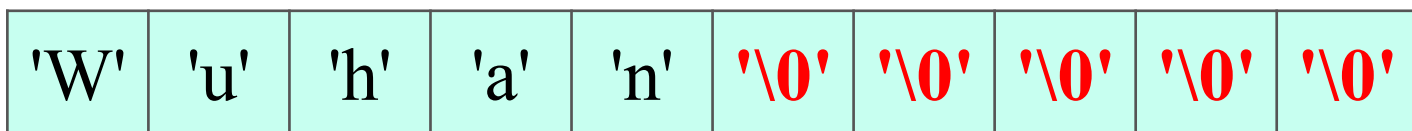
对字符数组初始化有两种形式：

1) 用初值表初始化：

如： `char s[ ]={'W','u','h','a','n'};`

- ◆ 没指定数组大小，自动计算数组大小：5个元素（字节）。**注：这个s不能用作字符串**（没有字符串结束标志'\0'）。

再如： `char s2[8]={'W','u','h','a','n'};`



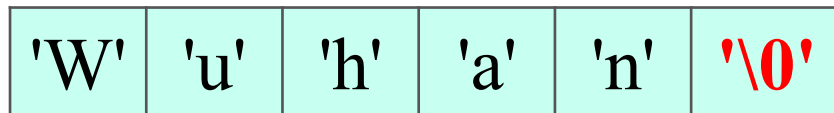
没有指定初值的单元被自动初始化为0

注：这种情况下，理论上可以作为字符串使用（末尾有0），但“初衷”不是字符串

2) 用字符串赋初值:

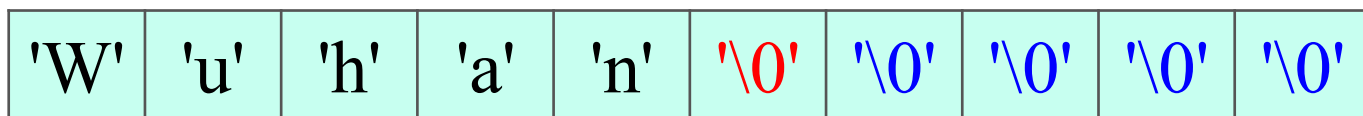
如: `char s3[ ]="Wuhan";`

- ◆ 系统将为s3自动分配**6个字节**: 存储字符串内容的**5个字符**+字符串结束标志**'\0'**。



再如: `char s4[12]="Wuhan";`

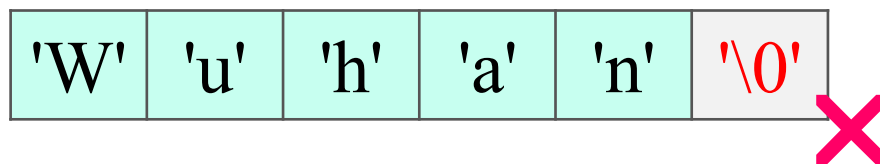
- ◆ 在s4的**前6个字节**中存储字符串内容和字符串结束标志'**\0**', 而多余的字节也被自动清零。



多余的单元被自动初始化为0

必须添加的0

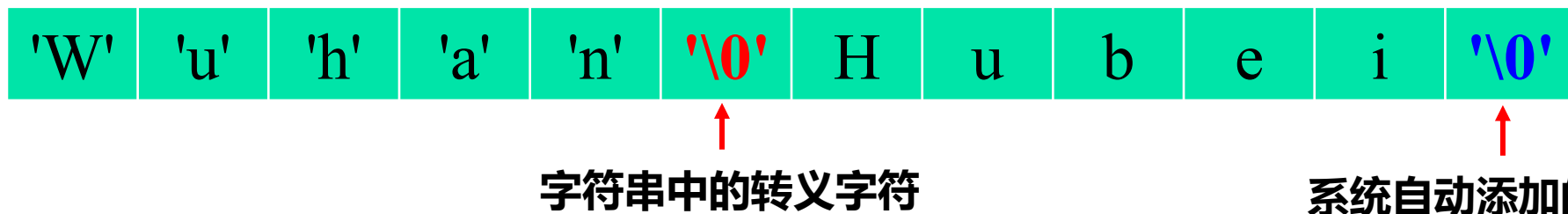
再如: `char s5[5]="Wuhan";`



指定的单元数目
不够用, 溢出!

再如: `char s6[ ]="Wuhan\0Hubei";`

↑
字符串中可以有转义字符



测试一下:

输出:

`printf("%d", sizeof(s6));`



12

根据双引号中的全部内容进行初始化

`printf("%s", s6);`



Wuhan

输出时以第一个'\0'作为字符串的结束

字符数组使用的要点：

- 如果是**普通的字符数组**，基本规则和其它类型的数组一样；
- 如果**用字符数组存储字符串**，要注意存储时最后位置上的**字符串结束标记'\0'**，其它和普通字符数组一样；

7.5.4 二维字符数组

1、二维字符数组

- **二维字符数组**就是以字符为元素的二维数组。

如： **char text[25][80];**

2、二维字符数组的初始化

■ 用初值表初始化

- `char s[2][4]={ 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'};`

- `char s[2][4]={ {'a', 'b', 'c', 'd'}, {'e', 'f', 'g', 'h'}};`

■ 用字符串初始化

- `char devices[3][12]={"hard disk", "CRT", "keyboard"};`

■ 省略第1维的方式

- `char devices[ ][12]={"hard disk", "CRT", "keyboard"};`

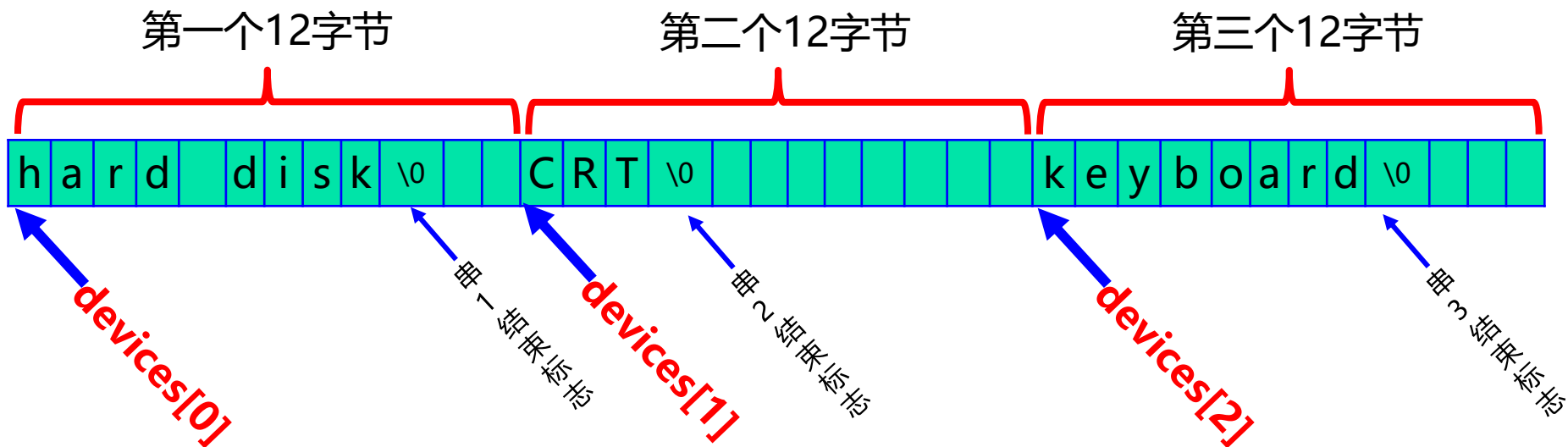
3、二维字符数组的存储结构 (行主序存储)

设有: `char devices[3][12]={"hard disk", "CRT", "keyboard"};`

二维字符数组的逻辑结构 (3行12列)

devices[0]	h	a	r	d		d	i	s	k	\0		
devices[1]	C	R	T	\0								
devices[2]	k	e	y	b	o	a	r	d	\0			

二维字符数组的存储结构 (行主序)



4、二维字符数组的使用 —— 字符串的数组

设有二维字符数组定义如下：

```
char weekend[2][10]={"Saturday", "Sunday"};
```

(1) 引用单个字符元素：二维数组名[第一维下标][第二维下标]

- ◆ 第 i 行第 j 列的单个字符： `weekend[i][j]`;

(2) 引用字符串：二维数组名[第一维下标]

- ◆ `weekend[i]` 可视为二维 `weekend` 数组中 i 行的名字，代表该行一维数组的首地址，也就是 i 行中的字符串的首地址；
- ◆ 输出 i 行的字符串： `printf("%s", weekend[i]);`

例7.25 字符串数组的输入输出操作举例

```
#include "stdio.h"
```

```
int main(void)
```

```
{    int i;
```

```
    char devices[3][12]={"hard disk","CRT","keyboard"};
```

```
    devices[0][0]= 'H';           /* "hard disk"变为"Hard disk"*/
```

```
    devices[2][0]= 'K';           /* "keyboard "变为"Keyboard "*/
```

```
    for(i=0;i<3;i++)
```

```
        printf("%s\n", &devices[i][0]);
```

i 行中首元素的地址也是
i 行中的字符串的首地址

```
    scanf("%s", devices[1]);
```

```
    for(i=0;i<3;i++)    printf("%s\n", devices[i]);
```

```
    return 0;
```

```
}
```


7.5.3 字符串处理函数

C语言提供了一些专门处理字符串的库函数。这些函数功能强大，要熟练使用。常用的有：

1. **strlen**：求字符串长度 (string length)
2. **strcpy**：字符串的拷贝 (string copy)
3. **strcmp**：字符串的比较 (string compare)
4. **strcat**：字符串的连接
5. **strstr**：求字符串的子串
6. **trim**：删除字符串首尾空白字符
7. **delete_c**：从字符串中删除所有与给定字符相同的字符
8. **reverse**：将字符串反转

这些标准函数都在头文件"**string.h**"中声明，因此使用这些函数之前首先要写上：**#include <string.h>**

如 调用字符串函数strlen求字符串的长度

```
#include <stdio.h>
```

```
#include <string.h>
```

使用字符串处理库函数前要包含头文件**string.h**

```
void main(void)
```

```
{
```

```
    char str[ ]="there is a boat on the lake";
```

```
    int length;
```

```
    length = strlen(str);
```

```
    printf("length of the string is %d\n", length);
```

```
}
```

运行结果：

length of the string is 28

自己实现字符串处理函数，学习字符串处理的方法

1、求字符串长度的函数 **strLen**

```
int strLen(char s[ ])
{
    int j=0;
    while(s[j] !='\0')
        j++;    /*j也是计数器*/
    return j;
}
```

```
#include <stdio.h>
void main(void)
{
    char str[ ]="there is a boat on the lake";
    int length;
    length = strLen(str);
    printf("length of the string is %d\n", length);
}
```

运行结果：

length of the string is 28

2、字符串复制函数 **strCpy**

字符串复制：将**源字符串 s** 的内容复制到**目的字符串 t** 中，包括最后的字符串结束标记'\0'。

```
void strCpy(char t[ ], char s[ ])  
{  
    int j=0;  
    while(t[j]=s[j++]);  
}
```



注：**赋值表达式的值是左操作数的值**，利用这个性质控制循环的执行：**为0的时候结束循环**。

```
#include <stdio.h>  
void main(void)  
{  
    char str1[30];  
    char str2[ ]= "there is a boat on the lake.";  
    strCpy(str1,str2);  
    puts(str1);    /*输出字符串*/  
}
```

运行结果：

there is a boat on the lake.

3、两字符串比较函数strCmp

字符串比较：对已知的两个字符串str1和str2，从它们的第一个字符起开始 **“成对比较”**。当两个字符相等时，继续比较下一对字符，直到遇到**第一对不相等的字符**或者到**第一个字符串的结尾**，算法结束。

返回值：退出点位置处**第一个字符串的字符减第二个串的字符**。

如：str1=“I am a teacher.”
 ↑-----↑
str2=“I am a student.”

不等：自左向右依次比较，找到第一对不相等的字符，返回 **'t'-'s'**。

再如：str1=“I am a teacher.”
 ↑-----↑
str2=“I am a teacher.”

相等：自左向右依次比较，一直到str1最后的'\0'处，此时 **'\0'-'0'=0**，代表都相等。

```

int strCmp(char s[ ], char t[ ])
{
    int j=0;
    while(s[j]==t[j] && s[j]!='\0')
        j++;    /*j是扫描下标*/
    return s[j]-t[j];
}

```

注：返回前即使s[j]==0，也计算s[j]-t[j]。

```

int main(void)
{
    char s1[ ]="car", s4[ ]="car";
    if(strCmp(s1, s4) > 0)
        printf("s1 greater than s4.\n");
    else if(strCmp(s1, s4) < 0)
        printf("s1 less than s4.\n");
    else printf("s1 equals to s4.");
    return 0;
}

```

运行结果：
s1 equals to s4.

如：str1="I am a teacher"

 str2="I am a teacher of HUST."

不等：第一个字符串到 '\0' 处和第二个字符串的字符不相等了。

strcmp的返回值：串中退出点位置处的两个字符的差值（即两个字符的ASCII码的差），**结果是整数**。

两个字符串s1和s2用strcmp比较，大小关系如下：

- ◆ 如果返回结果等于0，则代表两个**字符串相等**；
- ◆ 如果大于0，则称**s1大于s2**；
- ◆ 或者小于0，则称**s1小于s2**。

字典序

如：

```
int main(void)
```

```
{
```

```
    char s1[]="abc",s2[]="abd",s3[]="abc",s4[]="abb";
```

```
    printf("%s is %s %s.\n", s1, strCmp(s1,s2)>0 ? "greater than" :  
        strCmp(s1,s2)<0 ? "less than" : "equal to", s2);
```

```
    printf("%s is %s %s.\n", s1, strCmp(s1,s3)>0 ? "greater than" :  
        strCmp(s1,s3)<0 ? "less than" : "equal to",  
    s3);
```

```
    printf("%s is %s %s.\n", s1, strCmp(s1,s4)>0 ? "greater than" :  
        strCmp(s1,s4)<0 ? "less than" : "equal to", s4);
```

```
    return 0;
```

```
}
```

运行结果：

abc is less than abd.

abc is equal to abc.

abc is greater than abb.

C的字符串处理库函数中有三个用于字符串比较的函数：

(1) int **strcmp**(char *string1, char *string2);

- ◆ 比较时，区分英文字母的大小写

(2) int **stricmp**(char *string1, char *string2);

- ◆ 比较时，不区分英文字母的大小写

(3) int **strncmp**(char *string1, char *string2, int count);

- ◆ 只比较两个字符串的前面count个字符，其余字符不比较。

4、字符串连接函数 **strCat**

字符串连接：将一个字符串 **s** **连接**到另一个字符串 **t** 的后面。

方法：首先找到 **t 串的结束符 '\0'**，然后从这个 '\0' 所在的位置开始，把串 **s** 的字符依次存到 **t** 的后面，包括 **s** 的结束符。

```
char * strCat(char t[],char s[])
{
    int j=0, k=0;
    while(t[j++]!='\0'); ①找t的末尾
    j--; ②定位'\0'所在的位置
    while(t[j++]=s[k++]);
    return t; ③将s复制到t的尾部
}
```

```
void main(void)
{
    char s1[80]="I like ";
    char s2[ ]="the C programming.";
    strCat(s1,s2);
    printf("%s\n", s1);
}
```

这里有个空格

运行结果：

I like the C programming.

注：t要足够长，防止溢出。

5、求字符串子串的函数 **strStr**

子串：如果一个**字符串 t** 是另外一个**字符串 s** 的**一部分**，则称字符串 t 是字符串 s 的**子串**。

如：**"hello"**是**"hello world"**的子串。

判断是否为子串：

已知两个字符串 s 和 t，**在 s 里找 t 的第一次出现**。

- 如果 **t 在 s 中出现了**，则返回 t 在 s 中第一次出现的位置，即 **t 的首字符此时在 s 中的下标**，表示 t 在 s 中出现，是 s 的子串。
- **否则返回 -1**，代表 t 不是 s 的子串。

```

int strStr(char s[ ], char t[ ])
{
    int j=0, k;
    for( ; s[j]!='\0'; j++) /* j扫描s串, 对j指示的每个位置, 测试后面连续的若干字符是不是t中的字符*/
        if(s[j]==t[0] ){ /* 先确定s的当前字符s[j]是不是t的首字符t[0], 是则代表t可能会出现*/
            k=1; /*k是计数器, 同时用k扫描t串*/
            while(s[j+k]==t[k] && t[k]!='\0') /*在t串还没结束前, 匹配s和t对应位置上的字符是否相等*/
                k++; /* 如果相等则k加1, 然后继续匹配下一个位置*/
            if( k==strLen(t) ) /* 若连续匹配成功的字符数与串t长度相同, 则表示成功*/
                return j; /*匹配成功, 返回t在s中的起始下标 ( j 的值) */
        }
    return -1; /*没有找到子串, 返回-1, 思考: 能返回0吗? */
}

```

```
void main(void)
{
    char s1[80]="C is the most widely used programming language.";
    char s2[]="use";
    int i;
    printf("the sub_string's beginning position is %d\n", strStr(s1, s2));
}
```

运行结果:

the sub_string's beginning position is 21

6、删除字符串首尾空白字符的Trim函数(自学)

空白字符：空格符 (' ')、制表符 ('\t')、回车符 ('\r')、换行符 ('\n') 等被称为**空白字符**。

一般，如果空白符号出现在字符串的首、尾两端是**多余**的，

如： " a b \t c \t\t\t\t\n\n\r "。

前面多余的空格

后面多余的制表符、回车符、换行符

出现在中间的空白符不是多余的

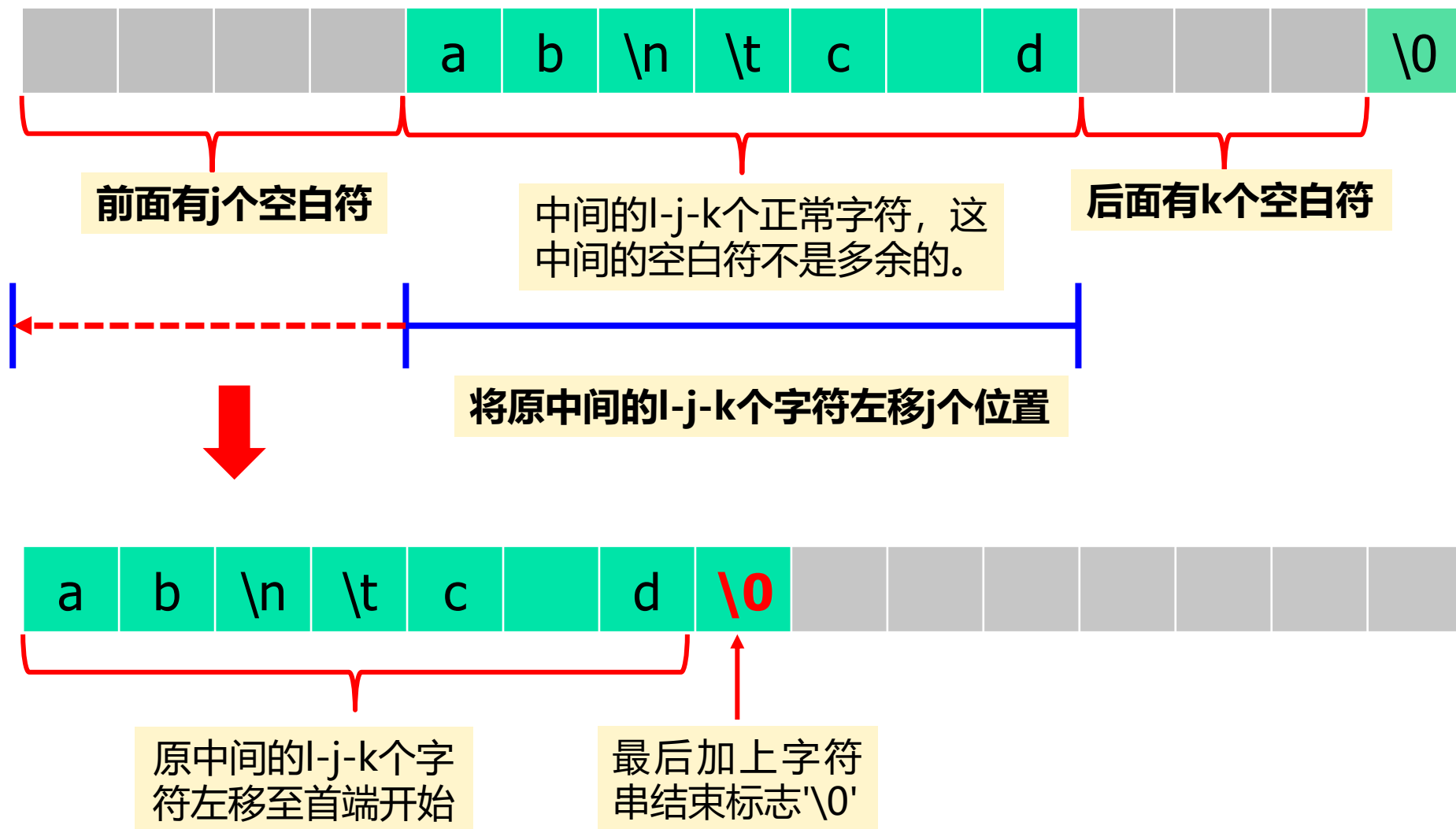
下面的程序可以删除一个字符串首、尾两端的空白符号：

(包括**空格符** (' ')、**制表符** ('\t')、**回车符** ('\r')、**换行符** ('\n') 四种空白符)

```

int Trim(char s[])
{
    int i, num, j=0, k=0, len=strlen(s);
    while(s[j]==' ' || s[j]=='\t' || s[j]=='\n' || s[j]=='\r')
        j++;    从前往后计算字符串前面的空白字符个数
    i=len-1;    /* 令i为字符串最后一个字符的下标*/
    while(s[i-k]==' ' || s[i-k]=='\t' || s[i-k]=='\n' || s[i-k]=='\r')
        k++;    从后往前计算字符串尾部的空白字符个数
    num=len-j-k; /*num等于串中的正常字符个数*/
    for(i=0; i<num; i++) s[i]=s[i+j]; /*将串中所有正常字符前移j个下标*/
    s[num]='\0'; /* 最后在s[num]处赋'\0', 形成字符串 */
    return strlen(s); /*返回去掉首尾两段空白字符后的串的长度*/
}

```



7、删除字符串中所有与给定字符相同的字符Delete_c (自学)

```
void Delete_c(char s[ ], char c)
```

```
{
```

```
    int j=0, k=0;    /*j读指示器, k写指示器*/
```

```
    while(s[j]!='\0') {
```

```
        if(s[j]!=c)
```

```
            s[k++]=s[j];
```

```
        j++;
```

```
    }
```

```
    s[k]='\0';
```

```
}
```

基本思想： 在字符串s的原地址空间内将不是要删除的字符前移，覆盖掉所有要被删除的字符即可。

在剩下的字符序列最后加上'\0'，形成字符串

8、将字符串反转的函数Reverse

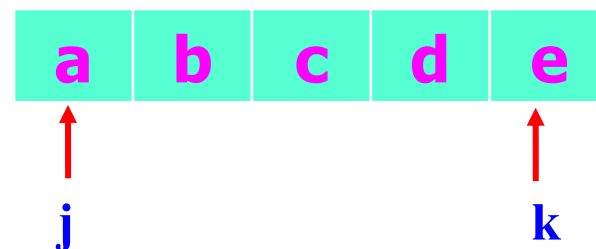
字符串反转：将一个字符串首尾颠倒过来。

如，将"abcde"颠倒为"edcba"

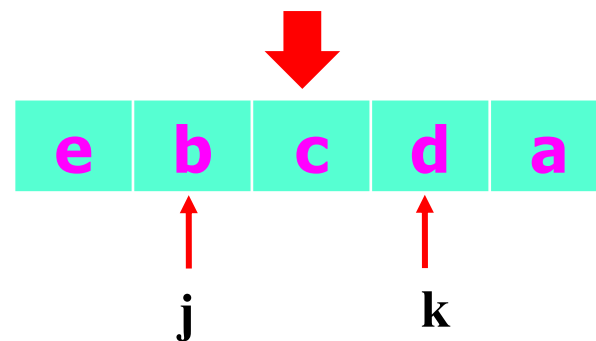
基本思想：前、后字符依次对调。

```
void reverse(char s[])
{
    int j, k;    /*j-前指示器 k-尾指示器*/
    char c;
    for(j=0, k=strlen(s)-1; j<k ; j++, k--)
        c=s[j], s[j]=s[k], s[k]=c;
}
```

逗号语句



当j和k还没相遇时，交换两个下标位置的元素



继续交换下一对元素直到 $i \geq k$

Trim, Delete_c, Reverse函数的应用

```
void main(void)
```

```
{
```

```
    char str[80]="  atbtctdtetft  ";
```

```
    printf("before trim, the string is \"%s\"\n", str);
```

```
    Trim(str);
```

```
    printf("after trim, the string is \"%s\"\n", str);
```

```
    Delete_c(str, 't');
```

```
    printf("after delete 't', the string is \"%s\"\n", str);
```

```
    Reverse(str);
```

```
    printf("after reverse, the string is \"%s\"\n", str);
```

```
}
```

运行结果：

before trim, the string is " atbtctdtetft "

after trim, the string is "atbtctdtetft"

after delete 't', the string is "abcdef"

after reverse, the string is "fedcba"

9、数字串与数之间转换的函数（参考例2.19自学）

数字串：由'0'~'9'等数字字符组成的字符串。如："54321"。

1) 数字串转换成数值

即将一个数字串表示的“多位数”转换成对应的数学意义下的值，如：将“54321”转换成 54321。

转换规则：

(1) 1个的数字字符转换成个位数：**数字字符 - '0'**

如：'0' - '0' = 0 '1' - '0' = 1

- ◆ 两个字符相减就是这两个字符对应的ASCII码相减。
- ◆ '0'~'9'在ASCII字符表中是连续排列的，所以一个**数字字符减 '0'** 就是其对应的数值。

(2) 多位的数字串转换成多位数

以十进制整数为例

多位的十进制数，每位都有一个权（第*i*位的权是 10^{i-1} ），

如，5位数54321：

权	10^4	10^3	10^2	10^1	10^0
各位 数码	5	4	3	2	1

整个数的值 = Σ (各位数码*相应位的权)

如： $54321 = 5 * 10^4 + 4 * 10^3 + 3 * 10^2 + 2 * 10^1 + 1 * 10^0$

多位的数字串转换成多位数

首先把数字串的各位数字字符转换成对应的数值；

然后按照“**乘权求和**”的方法计算数值即可。

如，5位数字串“**54321**”：

	'5'	'4'	'3'	'2'	'1'
	↓	↓	↓	↓	↓
	每位“数字字符”减'0'				
各位 数码	5	4	3	2	1
权	10^4	10^3	10^2	10^1	10^0

整个数的值 = Σ 各位的数*相应位的权

如：**54321** = **5*** **10^4** + **4*** **10^3** + **3*** **10^2** + **2*** **10^1** + **1*** **10^0**

= (((**5*****10** + **4**) * **10** + **3**) * **10** + **2**) * **10** + **1**

将一个十进制数字串转换成为对应整数的aTol函数

```
#define BASE 10
```

十进制基数为10

```
int aTol(char s[ ])
```

```
{
```

```
    int j=0, num=0;
```

```
    for(; s[j] != '\0'; j++)
```

```
        num=num*BASE + s[j]-'0';
```

```
    return num;
```

```
}
```

从高位到低位：**高位乘10，然后加下一低位**，得到的和再乘10，然后再加下一低位，直到个位为止。

将当前位的数字
符转换为数值

$$54321 = (((((0 * 10 + 5) * 10 + 4) * 10 + 3) * 10 + 2) * 10 + 1)$$

2) 数值转换成数字串

即将数学意义下的值转换成数字字符串的表示。如将**54321**转换成“**54321**”。

转换规则：

(1) **1位数转换成1位数字符**：**1位数 + '0'**

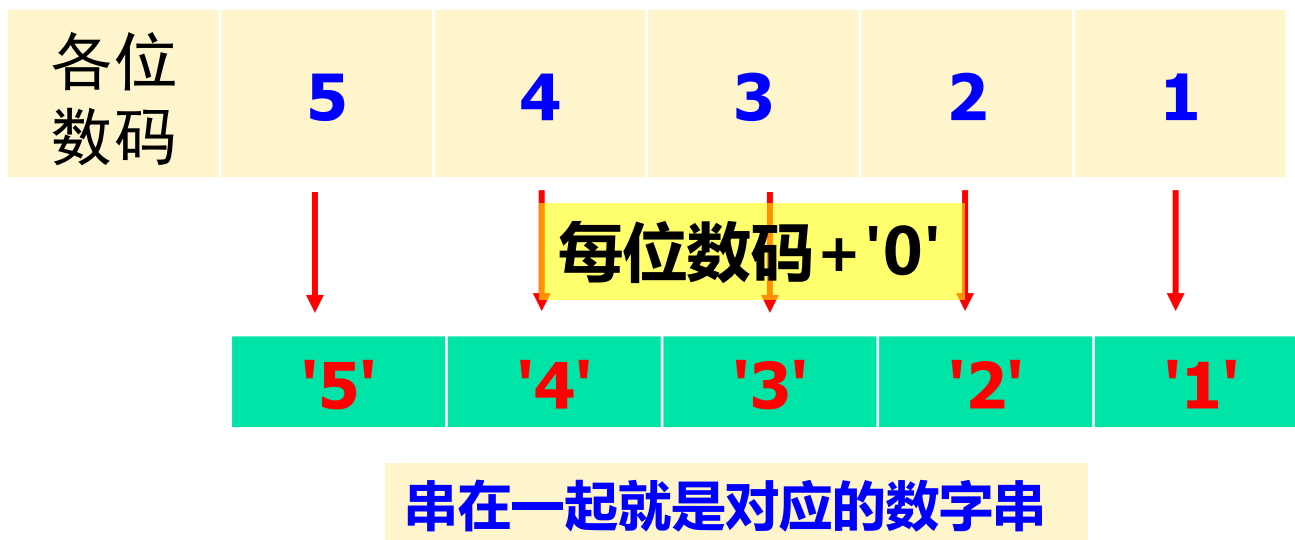
如：**0 + '0' = '0'** **1 + '0' = '1'**

即，**1位数+'0'** 就是其对应的数码字符。

(2) **多位数转换成多位数字串**

将多位数的**每一位提取出来**然后**加'0'**，就得到每位数码对应的数字字符，然后将这些数字字符从高位到低位“串”成一个数字串即可。

如，5位数 **54321**：



核心问题：怎么取得各位的数码？ **设整数n为一位N位数**，则

- ① **$n \% 10$** ，就得到个位的数码，如 $54321 \% 10 = 1$ 。
- ② **$n /= 10$** ，就得到一个N-1位数，再求新的 $n \% 10$ ，得到的就是原数中十位上的数码，如： $54321 / 10 = 5432 \% 10 = 2$ 。
- ③ 依次类推，直到到最高位为止，过程结束。

将一个整数转换成十进制的数字串 **iToa函数**

```
#define BASE 10    十进制基数为10

void iToa(int n, char s[ ])
{
    int sign, j=0;
    if((sign=n)<0)    /*sign处理符号位*/
        n = -n;    /*如果是负数, 先取反变为正数*/
    while(n>0) {    /*当n不等于0, 重复执行以下操作*/
        s[j++] = n%BASE + '0';    /*取模10的余, 并把数字转换成对应的数字字符*/
        n /= BASE;    /* "降低位数" */
    }
    if(sign<0) s[j++] = '-';    /*如果原数是负数, 在现在的数字串最后加负号*/
    s[j] = '\0';
    Reverse(s);
}
```

注：因为上述是从个位开始向高位依次处理并顺序保存到s数组中，所以s中的数字串是逆序的。为此**最后调用Reverse函数将其反转过来。**

如： $n = -54321$ 。

预备： $n = -n$ ，变成整数 $n = 54321$ ；

循环：

① $n \% 10 \rightarrow 1 + '0' \rightarrow s[0] = '1'$ ；并计算 $n /= 10 \rightarrow n = 5432$ ；

② $n \% 10 \rightarrow 2 + '0' \rightarrow s[1] = '2'$ ；再计算 $n /= 10 \rightarrow n = 543$ ；

③ $n \% 10 \rightarrow 3 + '0' \rightarrow s[2] = '3'$ ；再计算 $n /= 10 \rightarrow n = 54$ ；

④ $n \% 10 \rightarrow 4 + '0' \rightarrow s[3] = '4'$ ；再计算 $n /= 10 \rightarrow n = 5$ ；

⑤ $n \% 10 \rightarrow 5 + '0' \rightarrow s[4] = '5'$ ；再计算 $n /= 10 \rightarrow n = 0$ ；

循环结束。

此时得到的s数组中的内容是“12345”。因为原数是负数，所以在s的最后加'-'，得 $s = "12345-"$ 。最后颠倒过来得 $"-54321"$ ，转换完成。

例7.21 将一个十六进制数字串转换成对应整数的函数**hToi (自学)**

□ **hToi函数**：将一个存放在字符数组s中的**十六进制数字串**转换成为对应的**十进制整数**，返回转换后的整数。

□ **处理方法**：

(1) 十六进制下“各位”的权值为16，所以“**乘权求和**”为：

$$\text{num} = \text{num} * 16 + \text{当前位的数值}$$

(2) 一位的十六进制数字字符**s[j]**转换为对应的十进制数值：

- 若 $s[j] \geq '0' \ \&\& \ s[j] \leq '9'$ ，转换的数值为： $s[j] - '0'$ ；
- 若 $s[j] \geq 'a' \ \&\& \ s[j] \leq 'f'$ ，转换的数值为： $s[j] - 'a' + 10$ ；
- 若 $s[j] \geq 'A' \ \&\& \ s[j] \leq 'F'$ ，转换的数值为： $s[j] - 'A' + 10$ 。

```
int htoi(char s[])
```

```
{
```

```
    int j=0,num=0;
```

```
    if(s[j]=='0' && (s[j+1]=='x' || s[j+1]=='X')) j+=2;
```

```
    else return -1;    /*若前导符不是0x或0X, 不是合法的十六进制数字串*/
```

```
    for(; s[j]!='\0'; j++){
```

```
        if(s[j]>='0' && s[j]<='9')        num = num*16+s[j]-'0';
```

```
        else if(s[j]>='a' && s[j]<='f')    num = num*16+s[j]-'a'+10;
```

```
        else if(s[j]>='A' && s[j]<='F')    num = num*16+s[j]-'A'+10;
```

```
    }
```

```
    return num;
```

```
}
```

要求十六进制数的字符串必须以“0x”或“0X”开始，这里过滤前导符0x或0X



思考： **itoa**: 如何将十进制整数转换为十六进制的数字串？ 见教材例7.29

本章小结

- 一维数组的声明、初始化、存储结构和使用。
- 多维数组的声明、初始化、存储结构和使用。
- 字符数组的声明和使用，字符数据和字符串的关系，字符串处理函数，二维字符数组、字符串数组。
- 相关算法：矩阵加法、矩阵乘法，基于分治策略的二分查找，冒泡排序等。